

Intelligent systems for industry, supply chain and environment

LESSON 14

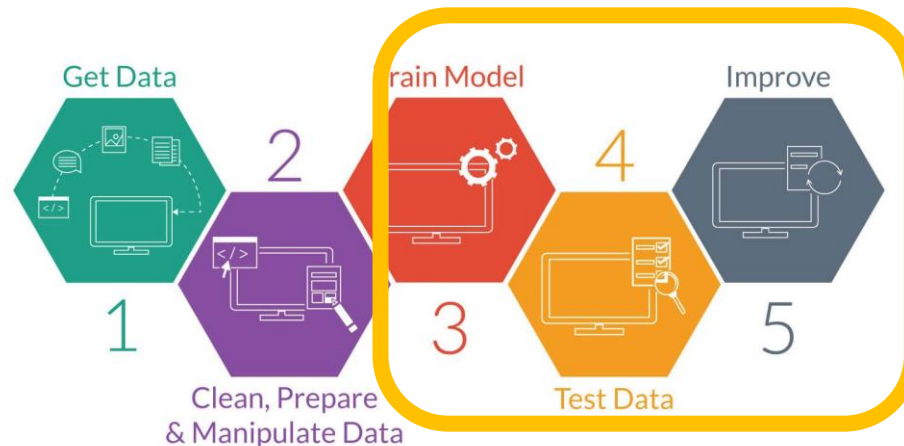
Workflow: optimization. Dataset balancing.
Dataset Partitioning. Accuracy assessment.



Outline

- L14 Workflow: optimization
- L14 Dataset Partitioning
- L15 Assessment of the accuracy of Regressors and Classifiers

Very important for the quality of your work

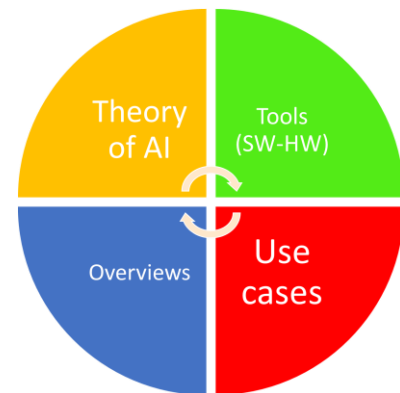




THEORY

Main concepts

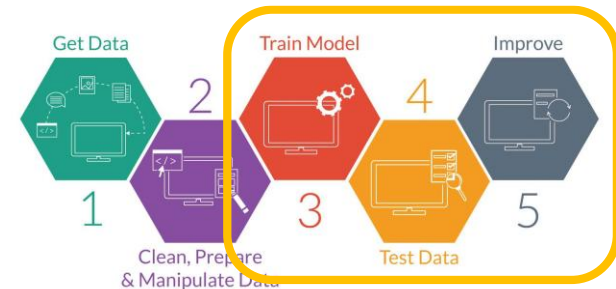
Just a brief recall from previous lessons



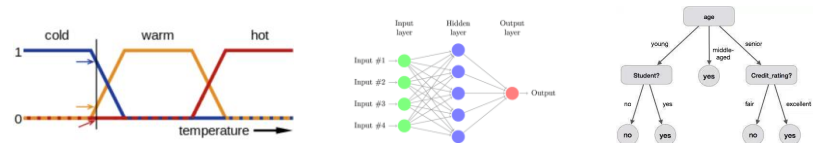
ML: the 3 main components

- As introduced in previous lessons, every machine learning algorithm has **3 components/steps**:

- **Representation** - #3
- **Evaluation** - #3, #4
- **Optimization** - #3, #5



- It's a framework to understand all intelligent systems, from basic classical model to deep learning systems.

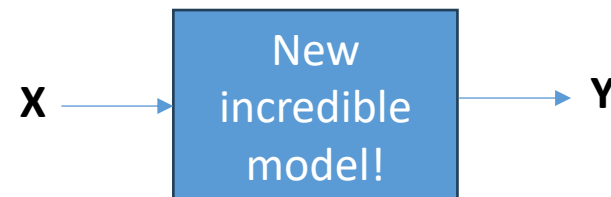
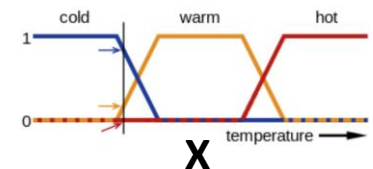
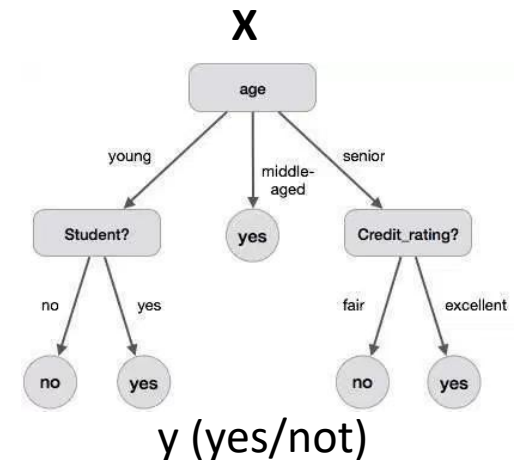
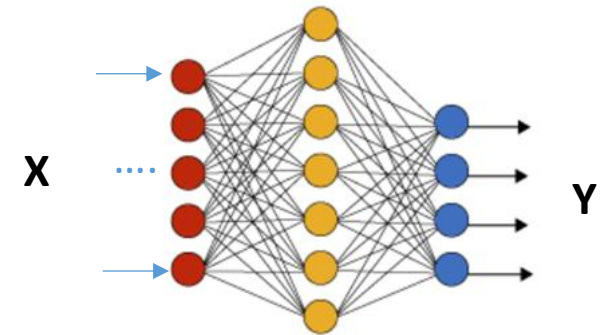


Representation

$$Y = \text{function}(X, \text{params})$$

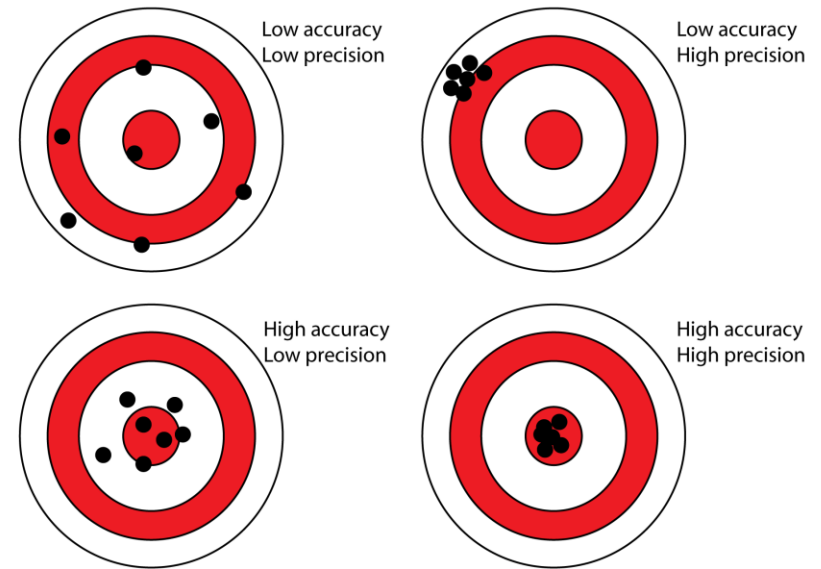
For a general model!

- Neural networks
- Decision trees
- Sets of rules / Logic programs
- Instances
- Graphical models (Bayes/Markov nets)
- Support vector machines
- Model ensembles
- Etc.



Evaluation

- **Accuracy/Precision**
 - Precision
 - Recall-Generalization
 - Squared error
 - Likelihood
 - Posterior probability
- ...



$$P(c|x) = \frac{P(x|c)P(c)}{P(x)}$$

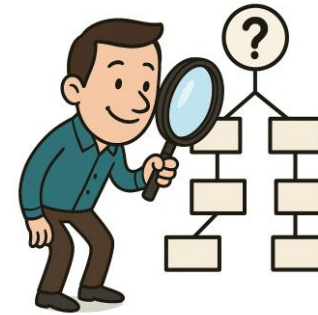
Diagram illustrating the relationship between the components of the equation:

- $P(c|x)$ is labeled as **Posterior Probability**.
- $P(x|c)$ is labeled as **Likelihood**.
- $P(c)$ is labeled as **Class Prior Probability**.
- $P(x)$ is labeled as **Predictor Prior Probability**.

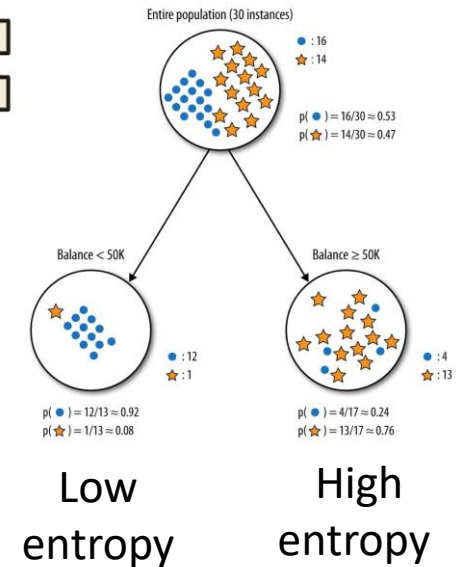
The goal of training is not just to reduce misclassifications, but to refine the model's estimation of posterior probabilities — the real foundation of intelligent decisions.

Evaluation

- ...
- Entropy
- Cost / Utility
- K-L divergence
- Margin (portfolio) See next slide
- Etc.



$$Entropy(t) = - \sum_{i=1}^C p_i \log_2 p_i$$



weighted output errors =

[ok ok ok **WRONG WRONG** ok ok]

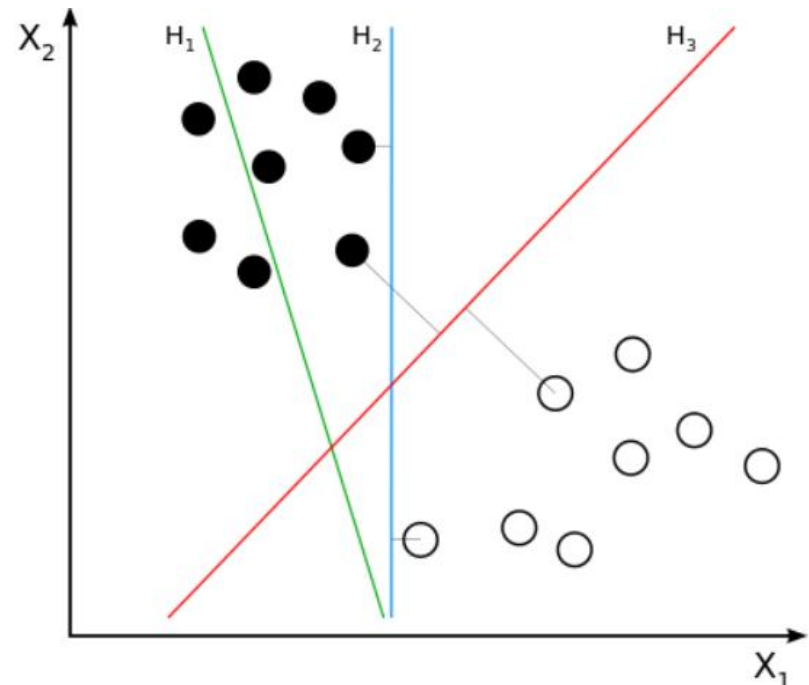
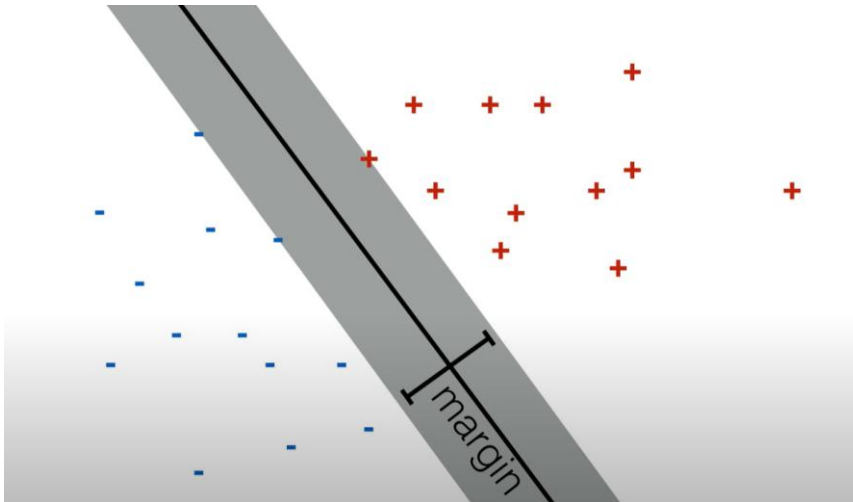
Evaluation: comparison between two figures

Accuracy VS Margin

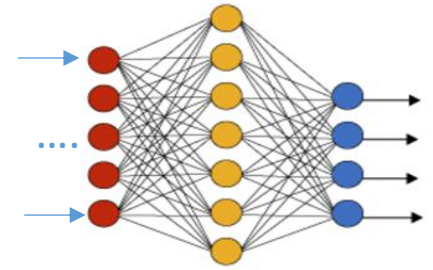
The decision boundaries H_2 and H_3 have both 100% accuracy (error = 0), but the margin values are different

We want to augment the margin!

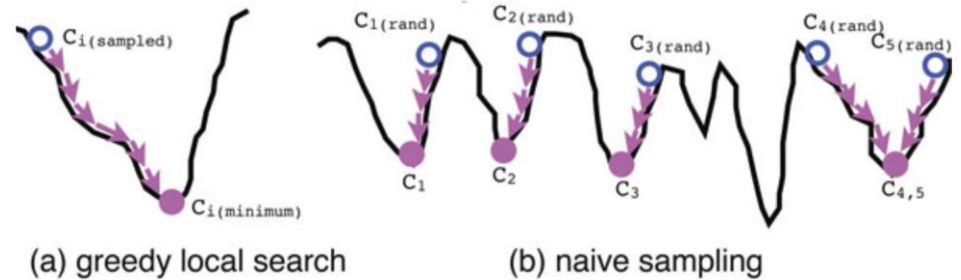
We prefer H_3



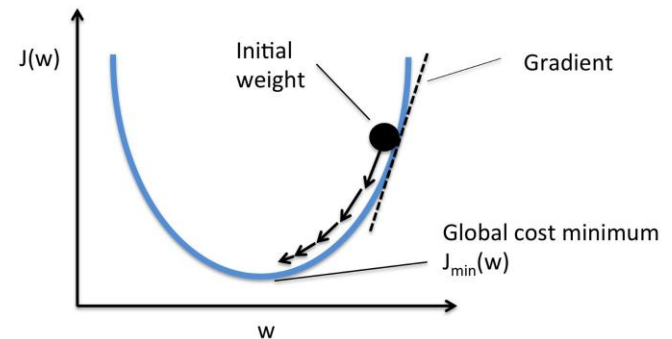
Optimization



- Combinatorial optimization
E.g.: Greedy search



- Convex optimization
E.g.: Gradient descent
- Constrained optimization
E.g.: Linear programming



Notation

$$Y = \text{function}(X, \text{params})$$

Vector
(multiple outputs)

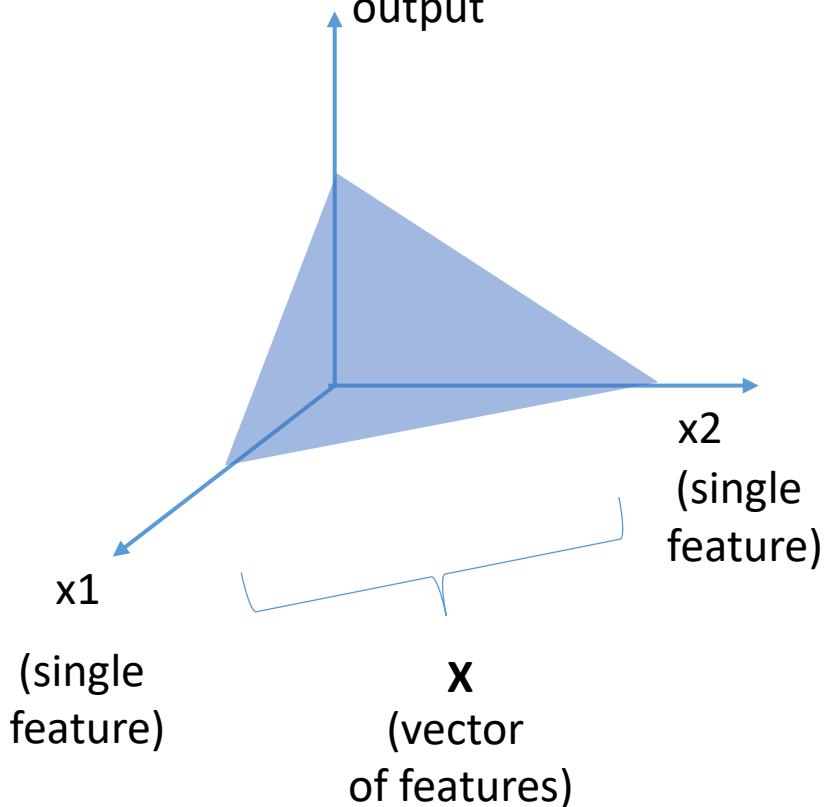
Vector
(multiple
inputs)

Vector
(multiple
params)

REGRESSION

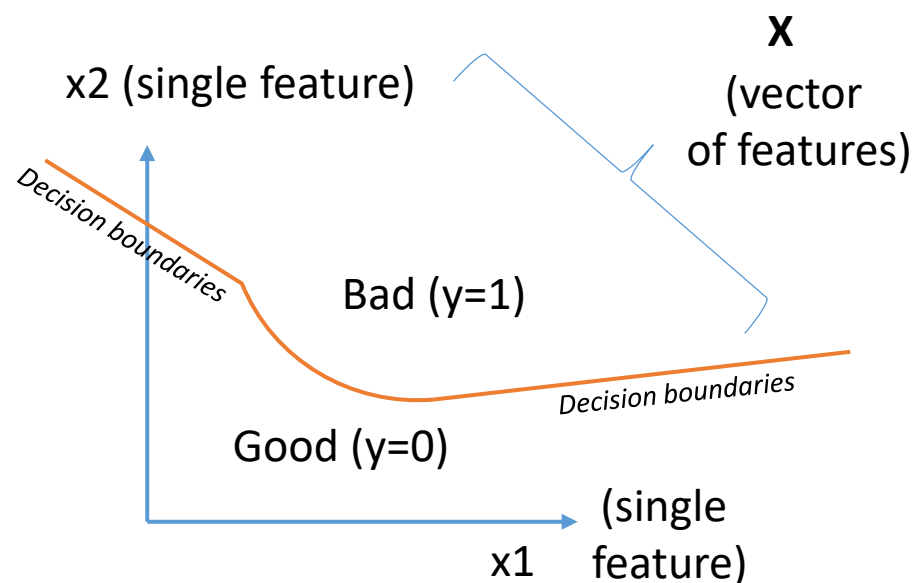
(explicit representation of y)

y (single value)
output



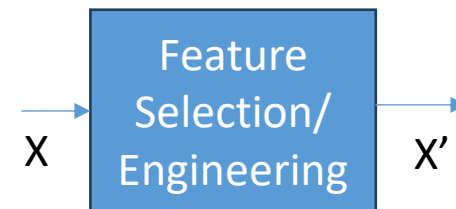
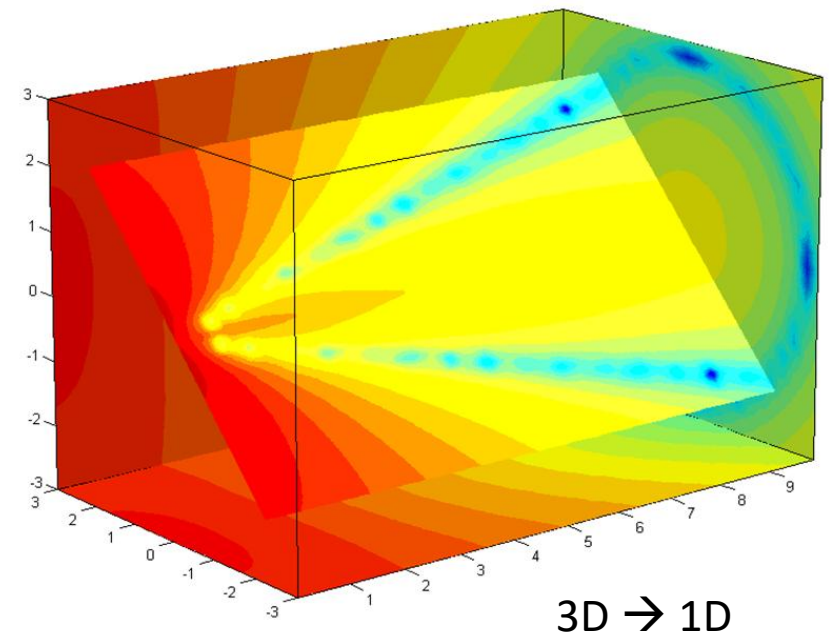
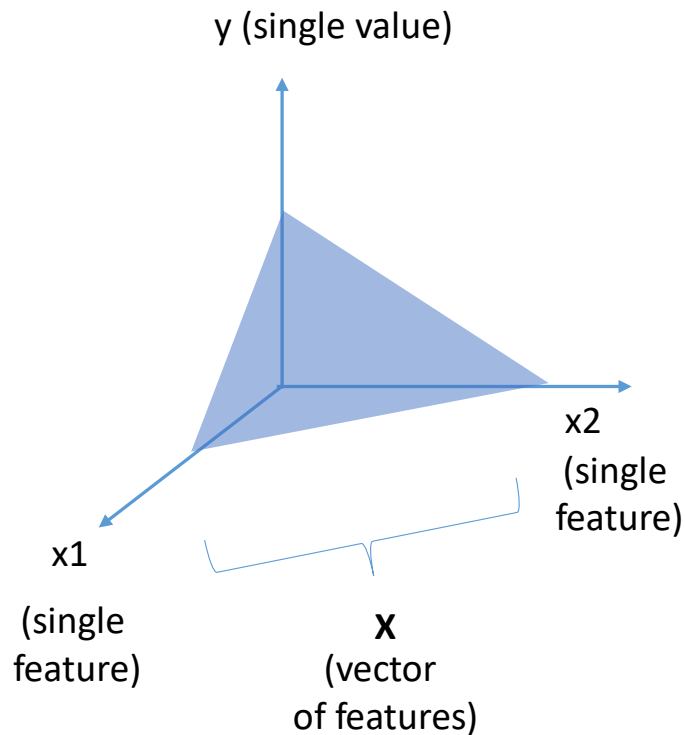
CLASSIFICATION

(implicit representation of y)



Regressor visualization

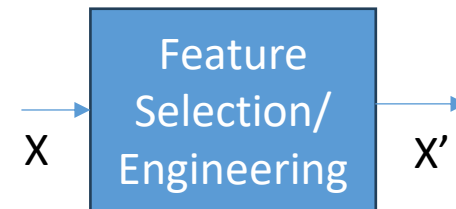
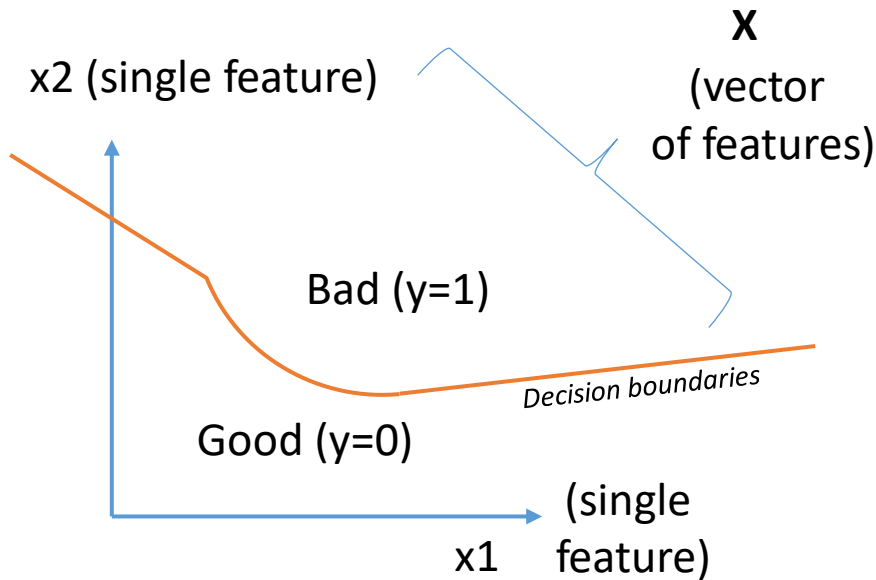
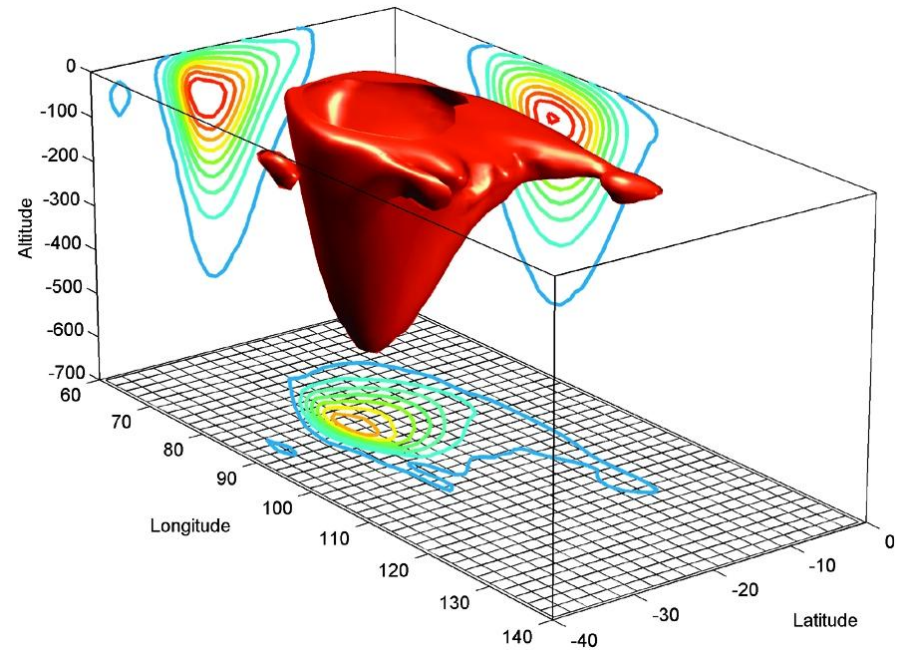
REGRESSION
(explicit representation of y)



We need to apply feature engineering and best visualization techniques to get valuable insights and perceive the behavior of our model

Classifier visualization

CLASSIFICATION
(implicit representation of y)



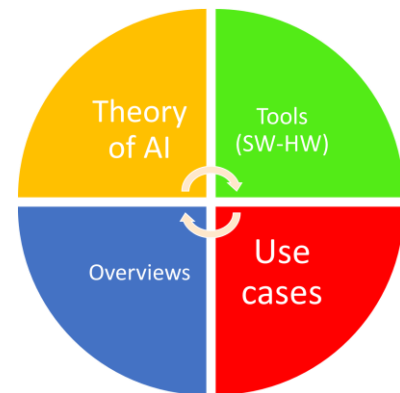
We need to apply feature engineering and best visualization techniques to get valuable insights and perceive the behavior of our model



THEORY

Dataset balancing

How to deal with odds distributions of classes



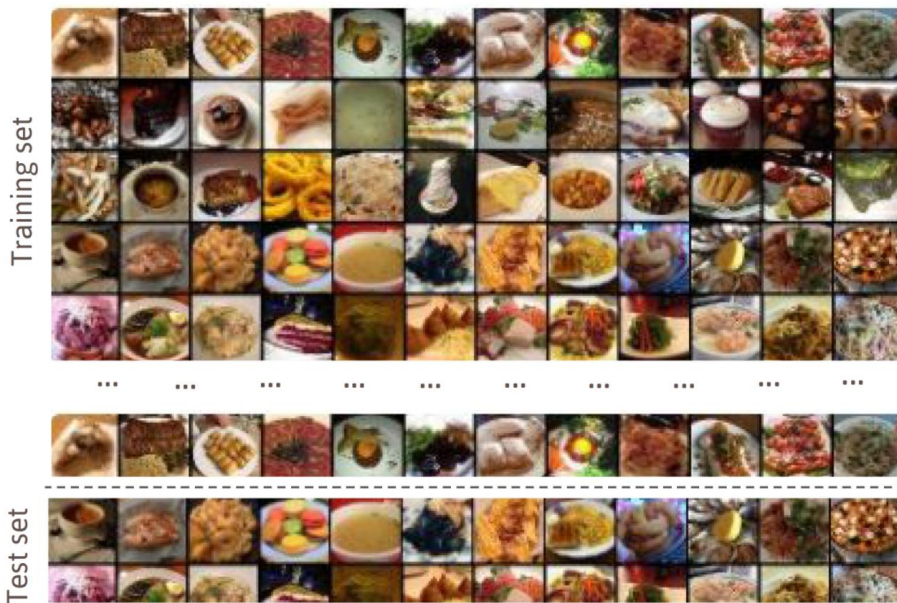
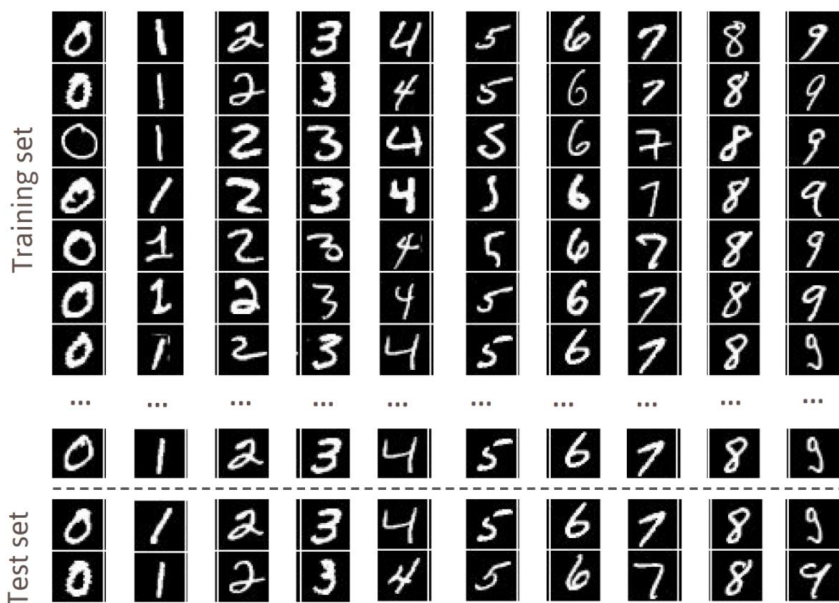
Data Sets balancing

MNIST is a very known dataset of handwritten digits containing approximately equally many samples of each digit.

Food 101 is another example of an academic dataset which contains exactly 1000 images of each of the 101 food categories.

In academia, the datasets are mostly balanced.

- That means for supervised classification problems there are usually equally many samples per class



Data Sets balancing: Unbalanced label distribution

- Many real-world databases are mostly unbalanced. Examples:
 - Medical vision: The majority of patients are healthy, while only a small fraction of the patients suffer from a certain disease.
 - Credit scoring: in fact, the majority of customers returns the loan while only 1–2% of people defaults.



Luckily, the **majority** of people are healthy



1-2% of people default

Handling Class Imbalance: Four Complementary Strategies

- **We introduce four complementary strategies** to handle class imbalance in security-oriented machine learning problems.
- Each strategy operates at a **different level of the ML pipeline (data, model, training)**.
 1. **More Data**
Collect additional samples for under-represented classes
 2. **Change Labeling**
Redefine, merge, or hierarchically organize classes
 3. **Sampling**
Modify the class distribution through over-sampling or under-sampling
 4. **Loss Weighting**
Penalize misclassification of rare classes more strongly during training
- **These strategies can be applied individually or combined** to address imbalance in our applied security use cases.

Data Sets balancing: unbalanced cost of missclassification

- Differently from many academic datasets, in many real-world unbalanced databases it is not correct to use a **cost of misclassification** equal for each class

	Paid	Defaulted
Accept		
Reject		

	Healthy	Sick
Diagnosed		
Not diagnosed		

Data Sets balancing solution 1: **More DATA!**

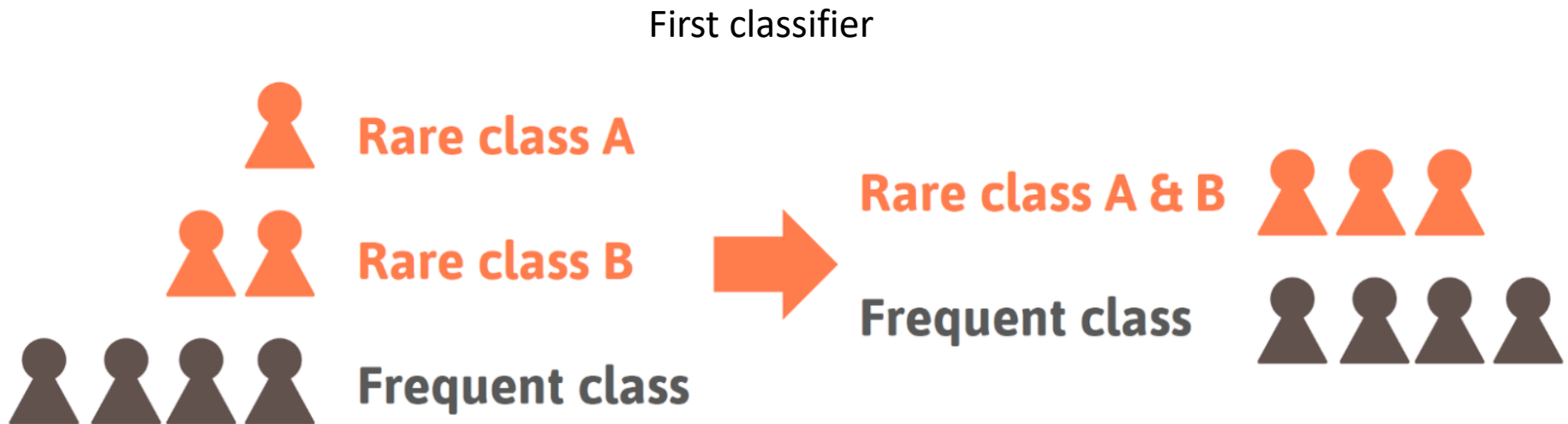
- You need to collect more data/images for rare classes
- Check the distributions (features and labels!) in train and validation/test!



Data Sets balancing solution 2:

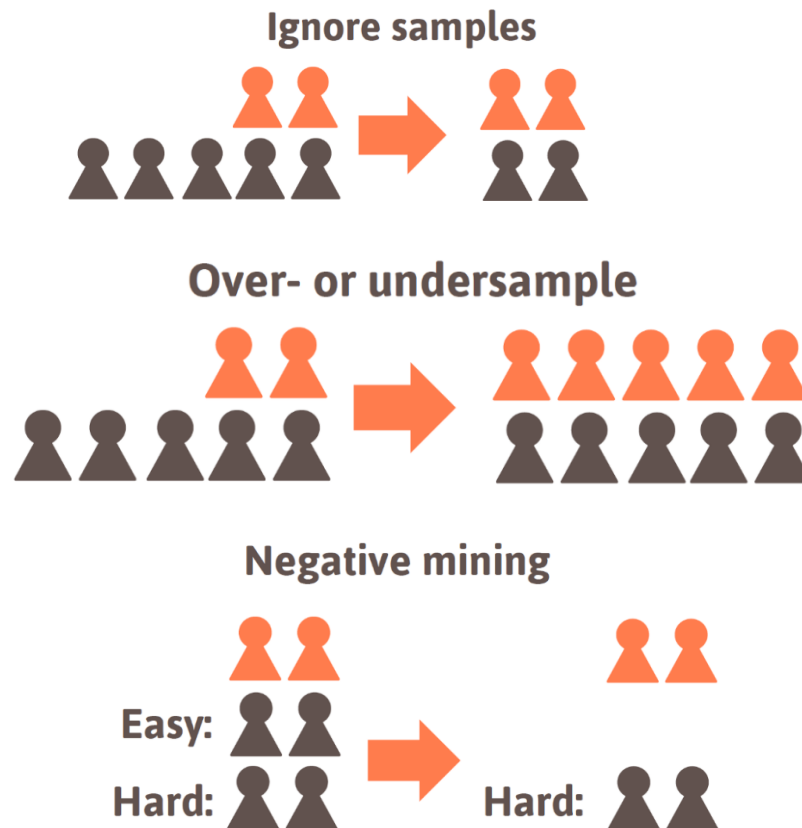
Change labelling

- Group and merge rare cases
- Divide et impera! → Hierarchical classifiers



Data Sets balancing solution 3: **Sampling**

- Do not use uniformly sampling methods

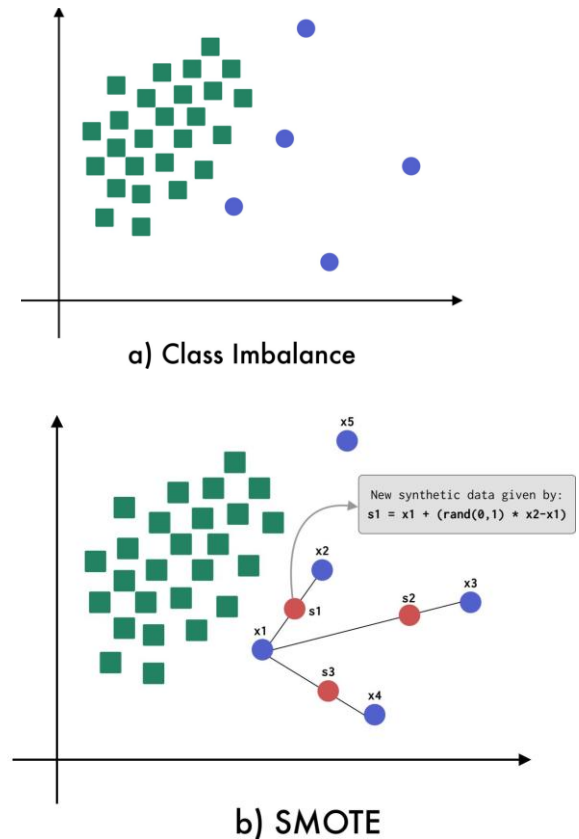


The advantage of this technique compared to the previous is that no samples are ignored.

Example: ask to the same patients to redo the tests

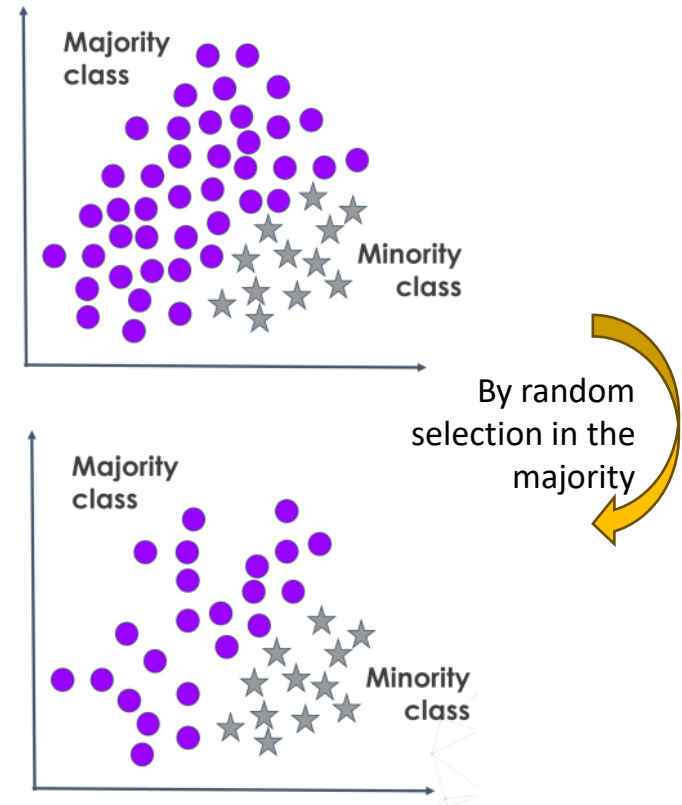
SMOTE – Synthetic Minority Over-sampling Technique

- SMOTE is a data-level sampling technique used to address class imbalance by generating *synthetic* samples of the minority class, instead of duplicating existing ones.
- For each minority sample x :
 - find its k nearest minority neighbors
 - generate a new sample along the line segment between x and one neighbor
- The feature space density of the minority class is increased



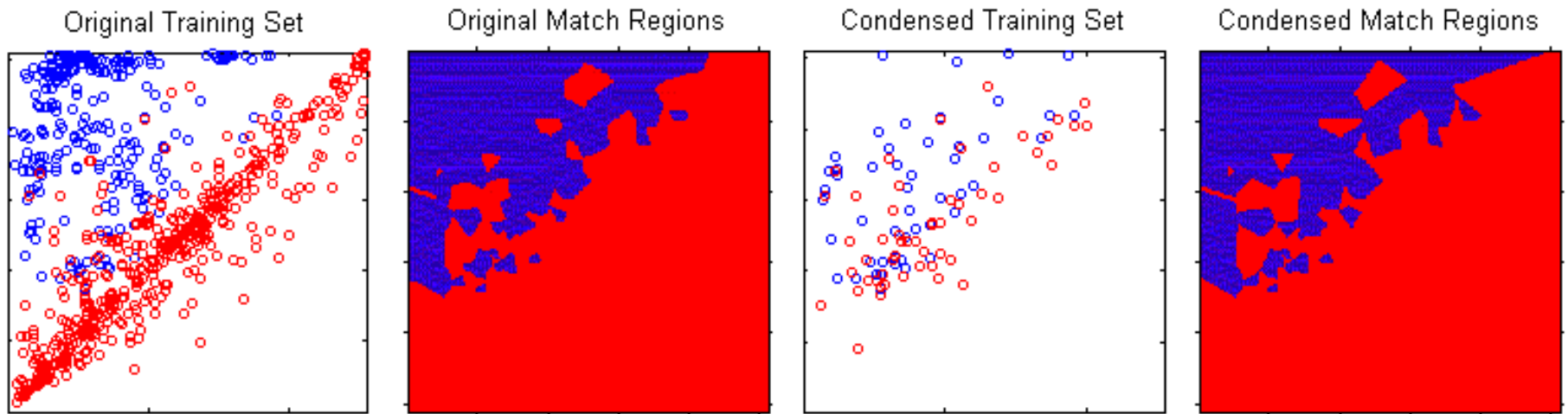
Undersampling with Condensed Dataset

- Undersampling with a condensed dataset aims to reduce class imbalance by **removing redundant majority samples**, while preserving the most informative data points for learning.
- The majority class often contains many **redundant and uninformative samples**.
- **Only samples close to the decision boundary are critical**
- A condensed dataset keeps:
 - all minority samples
 - a carefully selected subset of majority samples



Undersampling with Condensed Dataset

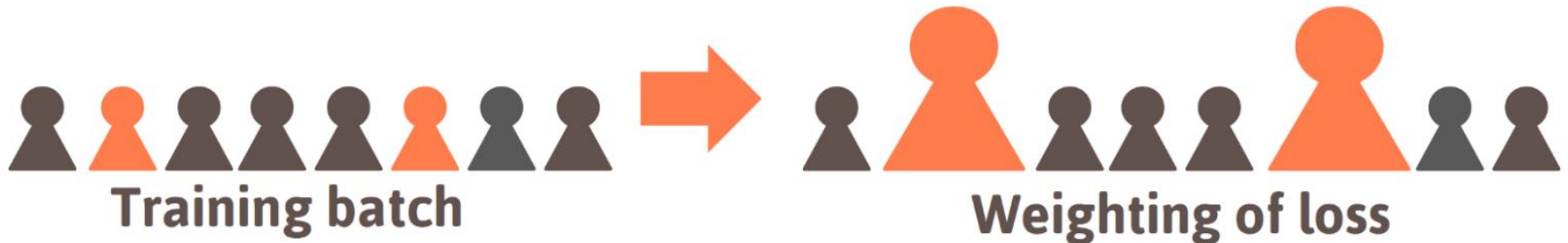
- Although many samples are removed, the local neighborhood relationships that define the k-NN decision boundary are preserved. As a result, the decision surface is not significantly altered.



Data Sets balancing solution 4:

Losses

- Increase the weight of the loss of samples from rare classes.

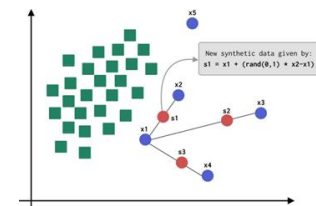


Dataset imbalance and related risks

These four strategies provide complementary ways to address class imbalance, each with different trade-offs and risks.

In practice, they are often **combined**, based on data availability, application constraints, and learning stability.

#	Strategy	Acts on	Typical Risk
1	More Data	Dataset	Cost / feasibility
2	Change Labeling	Problem definition	Loss of granularity
3	Sampling	Dataset	Overfitting / info loss
4	Loss weighting	Model	Training instability

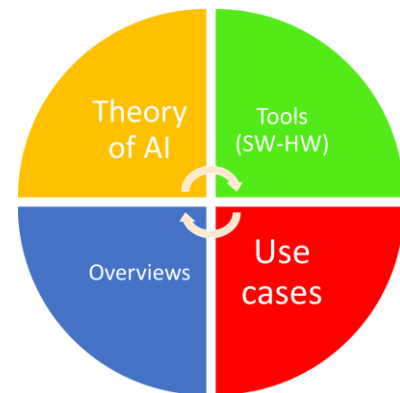




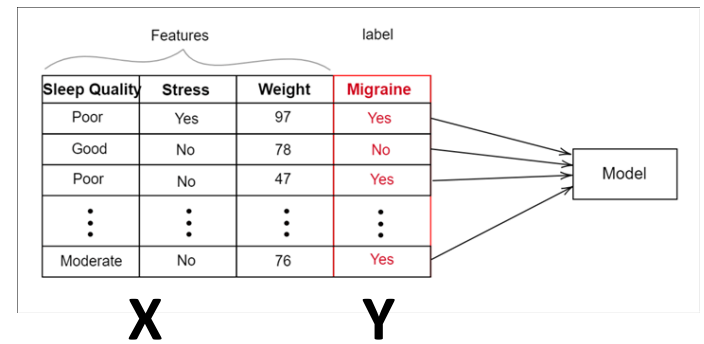
THEORY

Dataset partition

How to effectively train and test the models



Data Set of known $X \rightarrow Y$



- The benchmark data sets are used with one goal:
To evaluate the performance of a given model over a set of well-known standard problems

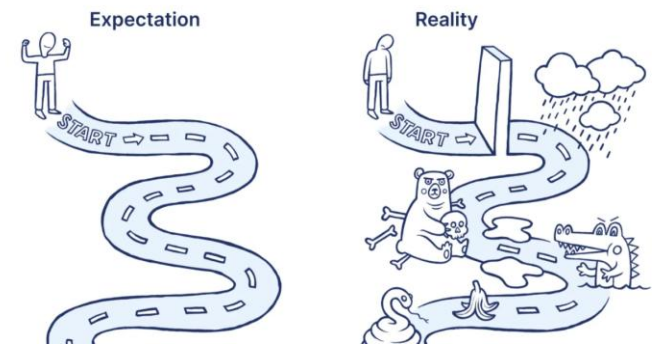
using dataset of known $X \rightarrow Y$

- However, the data must be correctly used in order to **avoid bias in the results**

Wrong Data Set Partitioning

Problem 1 → Optimistic bias

- **Optimistic bias** occurs when the evaluation is performed on data that influenced model design or selection, leading to an overly optimistic estimate of the true generalization error.
- If the **whole data set** is used for both build and then validate the model generated by a ML algorithm we have no clue about how the model will behave with new, unseen cases
 - This error estimation is the typical optimistic bias



Springer: Data Preprocessing in Data Mining

Wrong Data Set Partitioning

Problem 2 → Undetected Under/Over fitting

- Two main problems may arise by **using the same data** to train and evaluate the model (and you are not aware!):
 - **Underfitting** happens when the model is poorly adjusted to the data, suffering from high error both in training and test (unseen) data
 - **Overfitting** happens when the model is too tightly adjusted to data offering high precision to known cases but behaving poorly with unseen data.
- Note: a **proper data partitioning** method does not automatically solve these problems.
However, it helps you detect them and decide how to intervene.



Data Sets and Partitions

What can/can't be done

- The ultimate goal of any intelligent system is to be applied to real life problems
- Testing a technique in every problem is unfeasible!
- The common procedure is to evaluate such a technique in a set of standard problems/data sets publicly available



Data Set Partitioning: 1 critical warning!

In order to:

- control the model's performance
- avoid overfitting
- to have a generalizable estimation of the quality of the model obtained

several partitioning schemes are introduced in the literature

- How to partition the data is a key issue as it will largely influence in the performance of the methods
- Performing a bad partitioning will surely lead to incomplete and/or biased behavior about the model being evaluated.

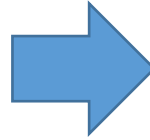
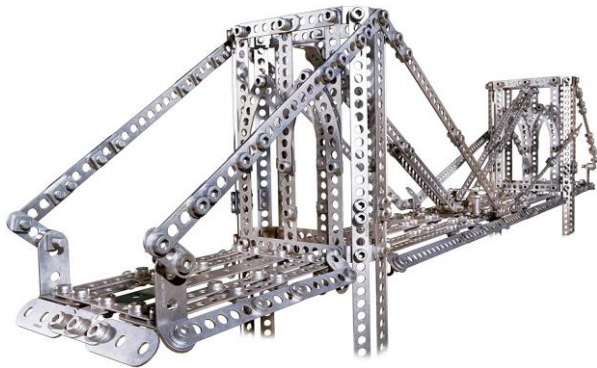


Focus now on...

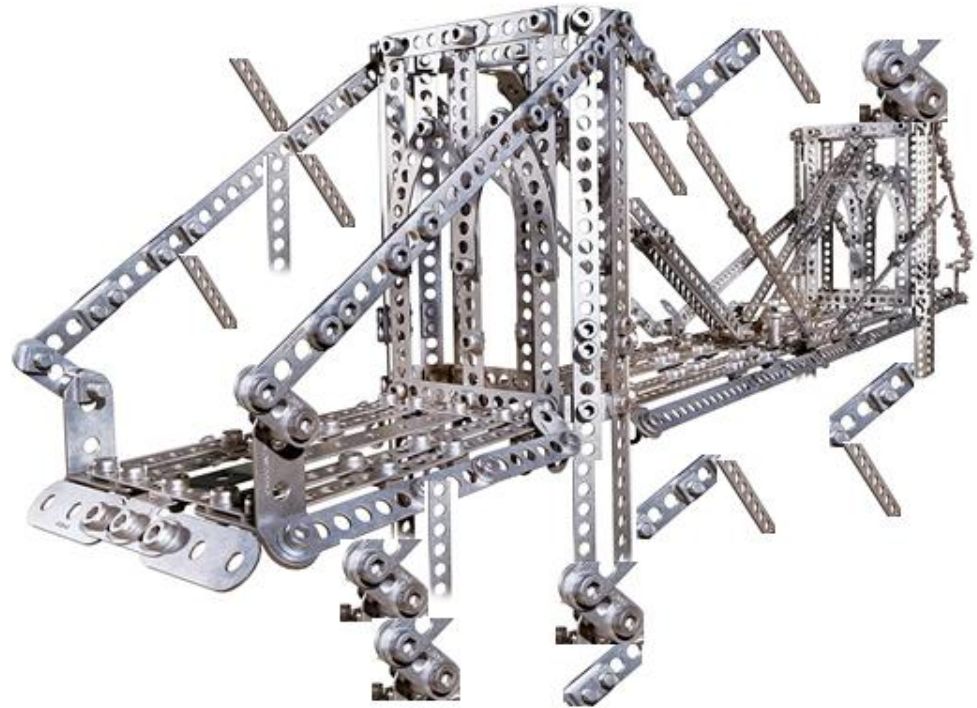


What if... Excessive #Par (elements) [RECALL]

Correct #Par
(after training)



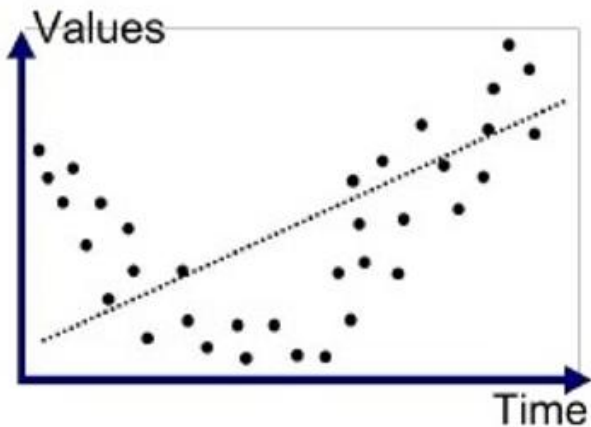
Excessive #Par
(after training)



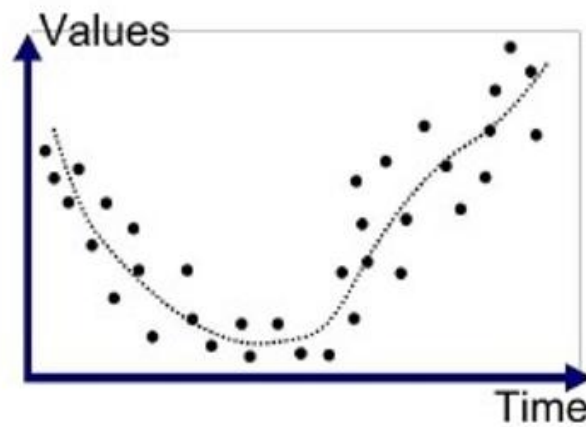
The learning method is not capable to deal with all the elements/parameters you inserted into the model with the given dataset!

The obtained model is **not optimal**, some parts or the model are **useless** or even you can get a **bad behavior**

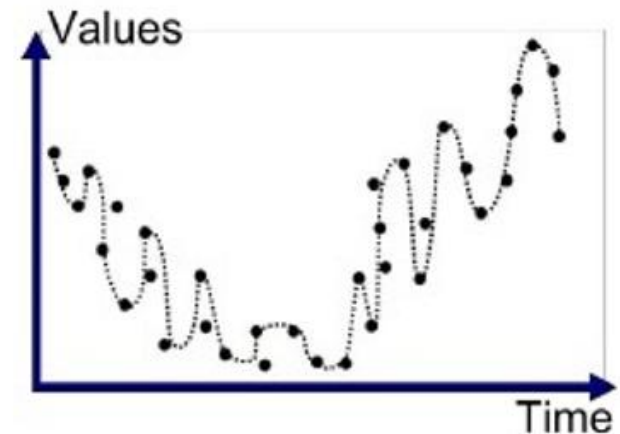
Overfitting 1D (Regressor model)



Underfitted

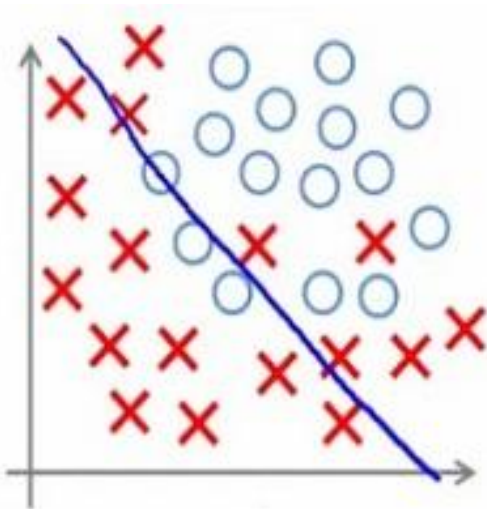


Good Fit/Robust



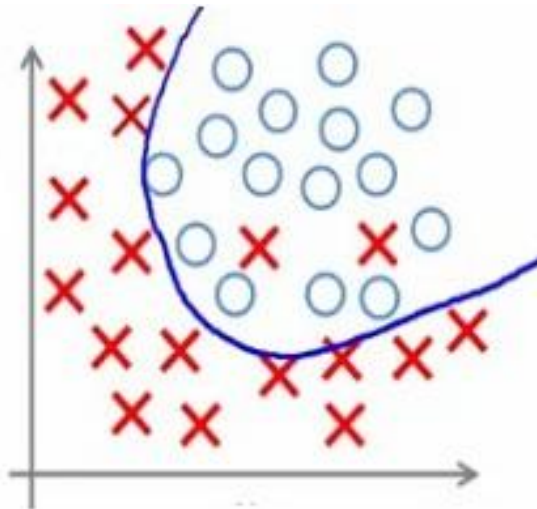
Overfitted

Overfitting 2D (Classification model)

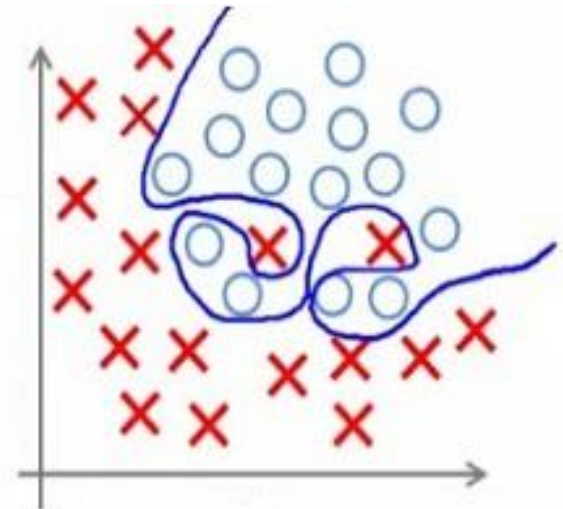


Under-fitting

(too simple to
explain the
variance)



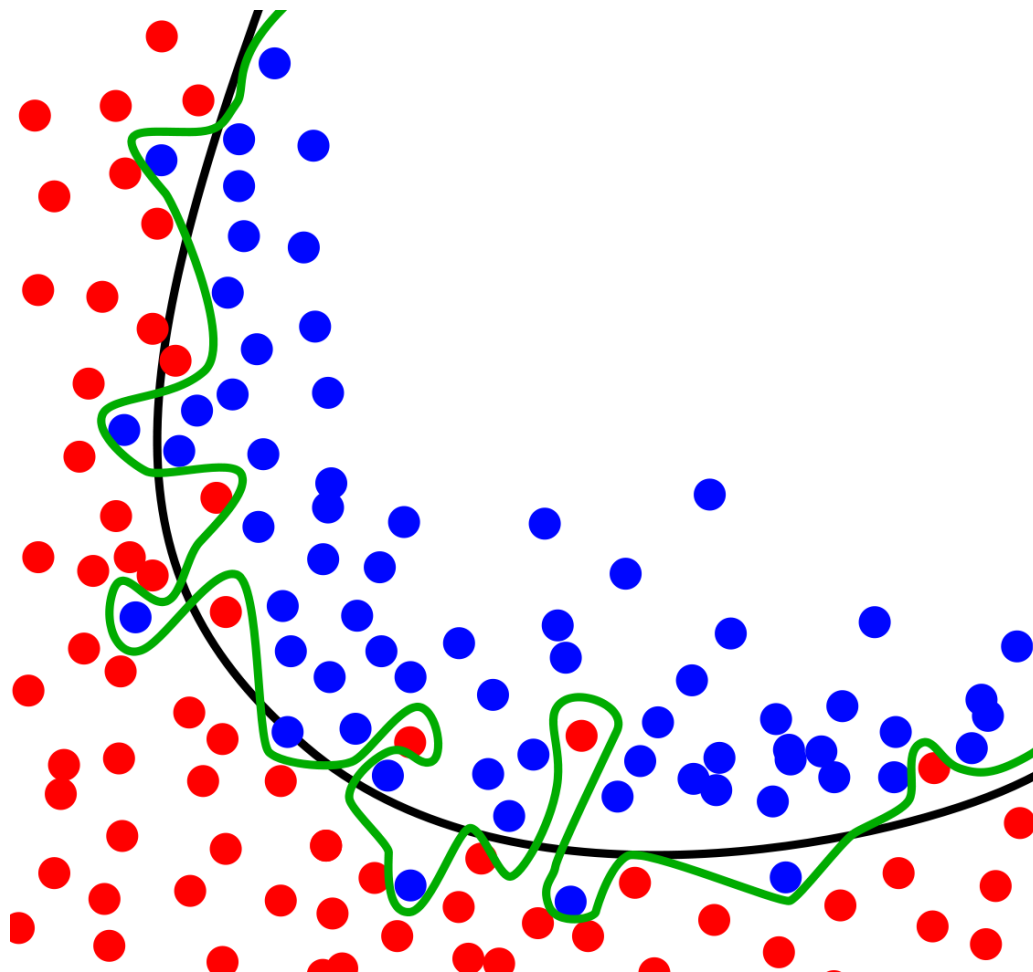
Appropriate-fitting



Over-fitting

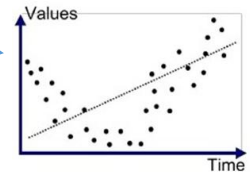
(forcefitting -- too
good to be true)

Overfitting 2D (bis)

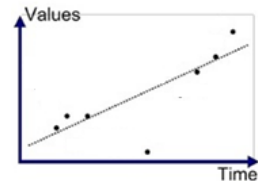


Data Set Partitioning: different cases

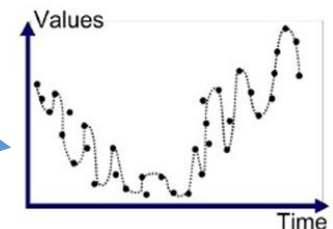
- **Underfitting** can be detected when a model shows **high error even on the training data**.
- The **lack of data** will also cause underfitting
 - The problem seems simpler than what is really is.
- **Adjusting the model** to better fit the data may lead to **overfitting**
 - the lack of unseen case makes impossible to notice this situation
 - Overfitting may also appear due other reasons like noise



Underfitted



Underfitted

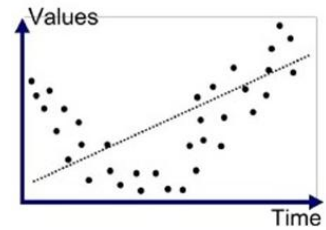


Overfitted

You are fitting the «noise in the signal»

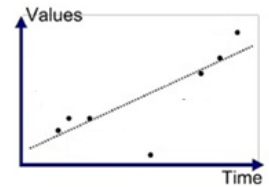
Data Set Partitioning

- By **using the whole data** we may be aware of underfitting problems due to a low performance of the model.



Underfitted

- The **lack of data** will also cause underfitting

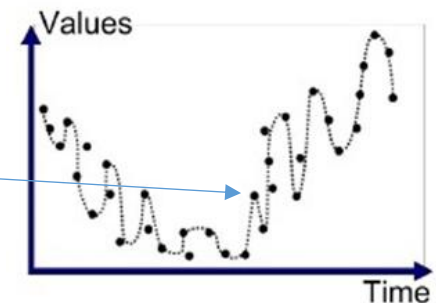


Underfitted

- **Adjusting the model** to better fit the data may lead to **overfitting**

- the lack of unseen case makes impossible to notice this situation
 - Overfitting may also appear due other reasons like noise

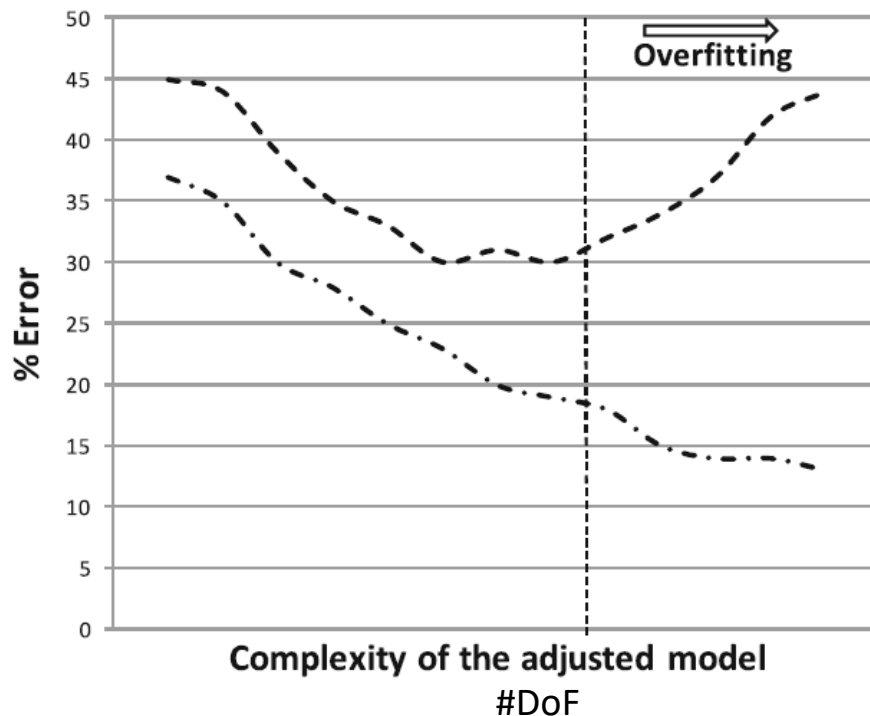
You are fitting the «noise in the signal»
not the relevant data



Overfitted

Data Set Partitioning (input space with high dimensionality)

- The input space can't be easily plotted, so you can study the errors to detect overfitting!



In this experiment we **redo** the training by of increasing the #DoF of the models (e.g., adding more neurons to a neural network)

This is not time, nor epochs

Splitting the data for Train and Test

It's a very
difficult
problem
/decision



Train/Test data partition Dilemma

The model
 $Y = \text{function}(X, \text{params})$

Train data



Test data

Lucky?
Real generality?
Statistical confidence?



New material!

Train/Test dilemma (2)

The model
 $Y = \text{function}(X, \text{params})$

Train data



Enough data to learn all cases?
Am I really exploiting my dataset?

Test data



Strong
confidence

New material!

Two cases

- **Too much training data, too little test data**

- good learning
- unreliable performance estimates
- risk of “lucky” test results



- **Too much test data, too little training data**

- reliable evaluation
- poor learningrisk of
- underfitting and missed patterns

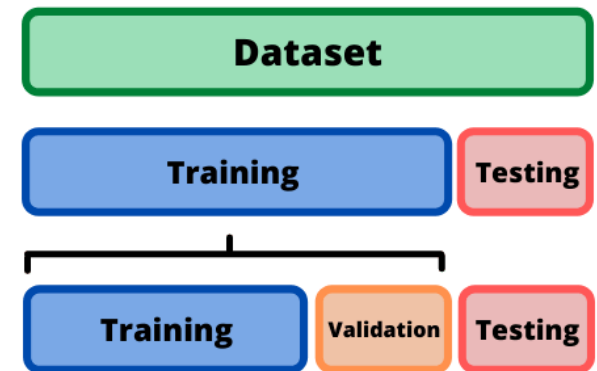


- **Core dilemma**

- learning accuracy **vs** evaluation reliability
- data efficiency **vs** statistical confidence

Training / Validation / Test

- Training
 - learn model parameters
- Validation
 - tune hyperparameters
 - select the model
- Testing
 - estimate generalization



Train to learn, validate to choose, test to evaluate.

Validation must part of the training set

VALIDATION CAN BE

Hyperparameter tuning

- learning rate
- network depth and width
- regularization strength (L2, dropout rate)
- batch size

Model selection

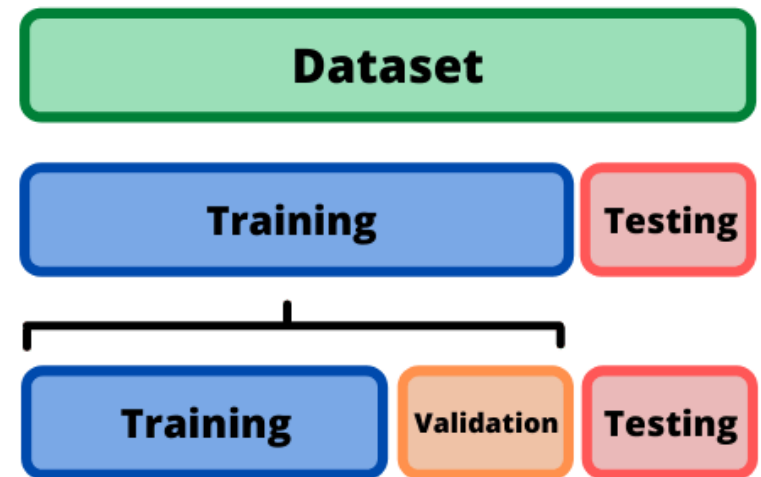
- choose between different architectures
- select the best-performing configuration
- compare alternative models

Early stopping

- monitor validation loss or accuracy
- stop training when validation performance degrades
- prevent overfitting to the training data

Training diagnostics

- detect underfitting vs overfitting



If you use the whole dataset for validation, you get an estimation affected by **optimistic bias**

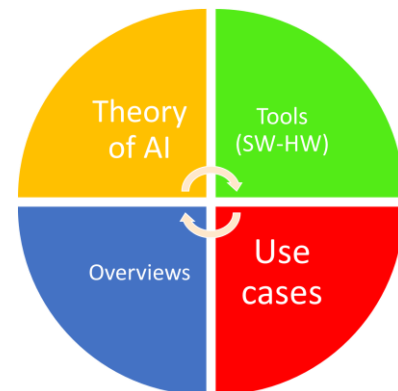


THEORY

Topology, Pruning, Regularization

Where to apply these techniques

**** A FUNDAMENTAL CONCEPT ****



All «decisions» must be made using just the train dataset

The model
 $Y = \text{function}(X, \text{params})$



Train data



Design activities:

- #of neurons
- #of layers
- Regularization
- Pruning
- Normalization
- PCA
- Feature engineering
- Etc..

Real Train data

Test data

USE IT ONLY FOR
GENERALIZATION ERROR

****NO DESIGN HERE****



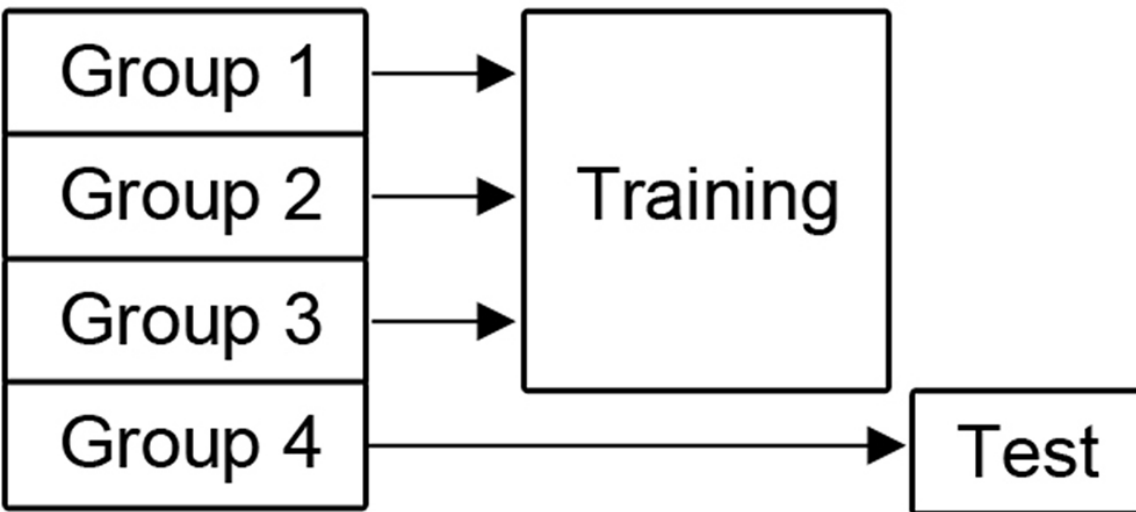
**Must keep it as
new material!**



k -Fold Cross Validation (k -FCV)

Cross validation general approach

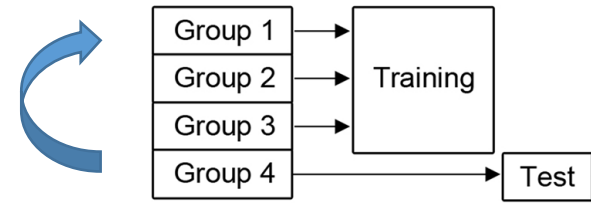
A Cross-validation



B Prediction testing

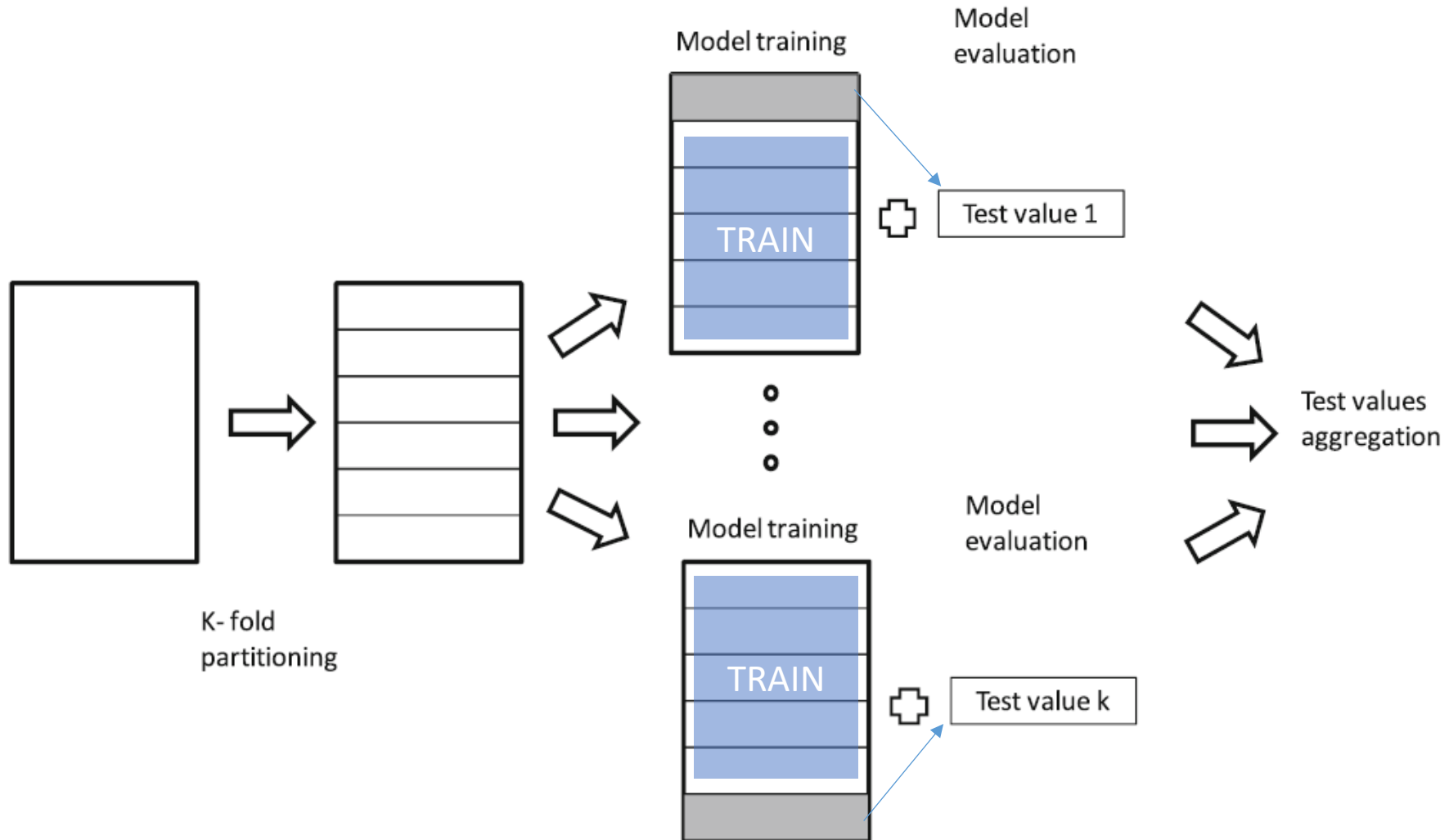


Data Set Partitioning: k-FCV



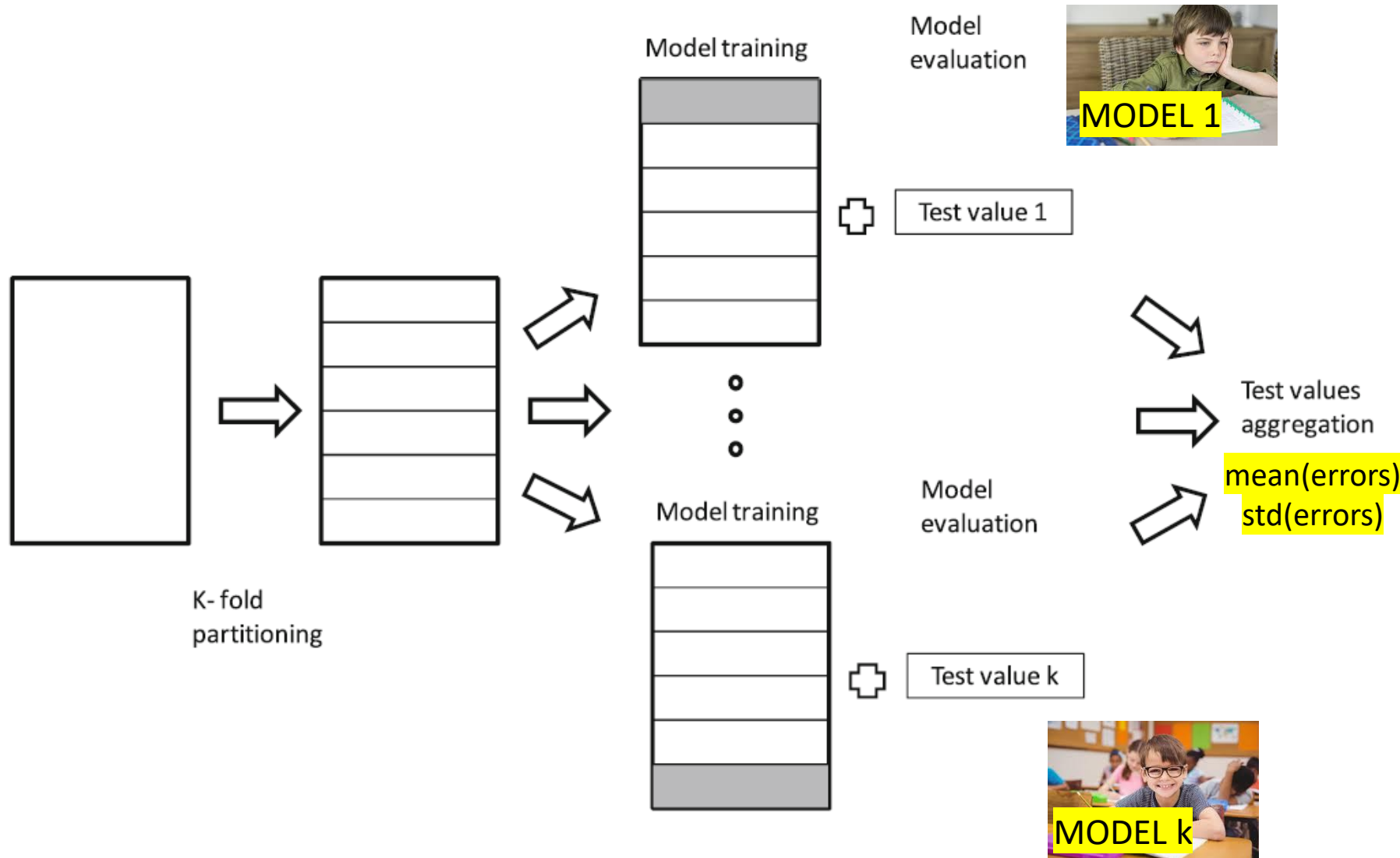
- The most common one is *k*-Fold Cross Validation (*k*-FCV):
 1. In *k*-FCV, the original data set is randomly partitioned into *k* equal size folds or *partitions*
 2. From the *k* partitions, one is retained as the validation data for testing the model, and the remaining *k* – 1 subsamples are used to build the model.
 3. As we have *k* partitions, the process is repeated *k* times with each of the *k* subsamples used exactly once as the validation data.

Data Set Partitioning: k-FCV



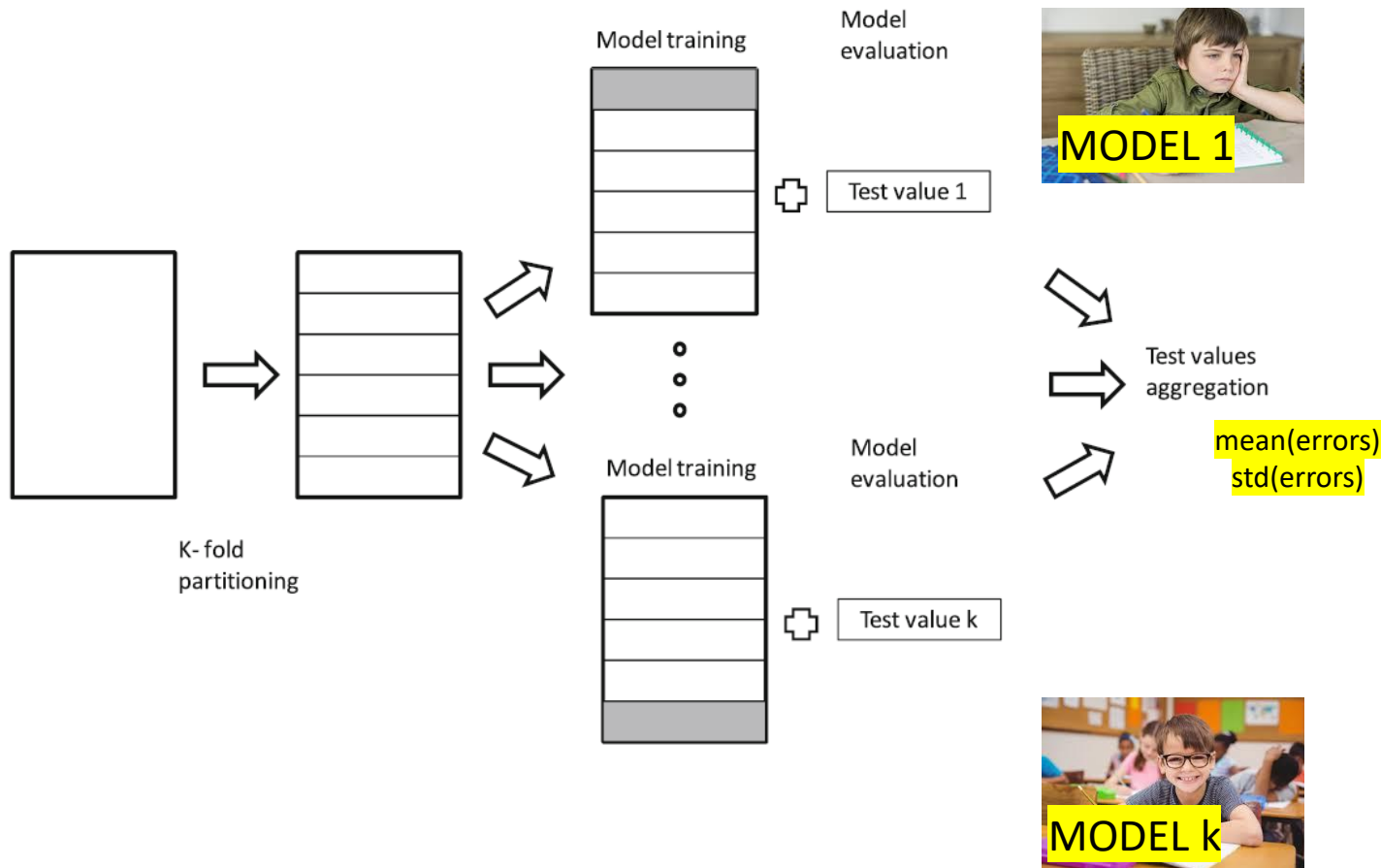
ONE IMPORTANT NOTE!!

Data Set Partitioning: k-FCV



ONE IMPORTANT NOTE!!

Data Set Partitioning: k-FCV



You trained
different models.
Which one to
deploy?

Data Set Partitioning: k-FCV

- The value of k may vary, 5 and 12 being the most common ones
- k needs to be adjusted to avoid to generate a small test partition poorly populated with examples that may bias the performance measures used.
 - If big data sets are being used, 10-FCV is usually utilized
 - For smaller data sets 5-FCV is more frequent

Data Set Partitioning: k -FCV

- Simple k -FCV may also lead to disarranging the proportion of examples from each class in the test partition
- The most commonly employed method in the literature to avoid this problem is **stratified k -FCV**
 - It places an equal number of samples of each class on each partition to maintain class distributions equal in all partitions

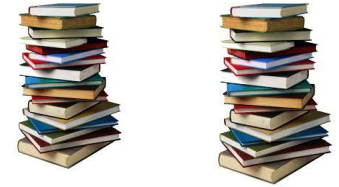


5×2 Cross Validation (5×2 CV)

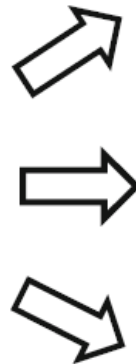
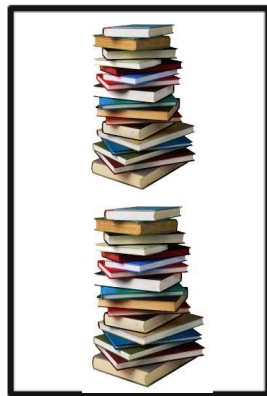
Data Set Partitioning: 5×2 CV

- The dataset is randomly divided into two equal halves (50% training, 50% testing).
- A model is trained on one half and tested on the other.
- The process is reversed, meaning the other half is used for training, and the first half is used for testing.
- This two-way split (2CV) is repeated 5 times, leading to a total of 10 evaluation scores.

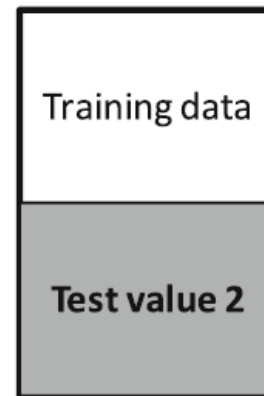
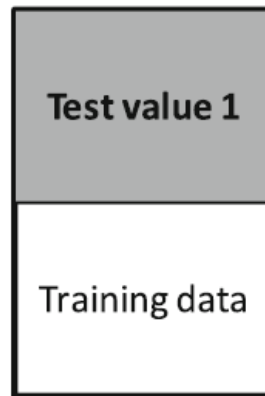
Data Set Partitioning: 5×2 CV



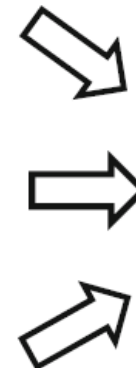
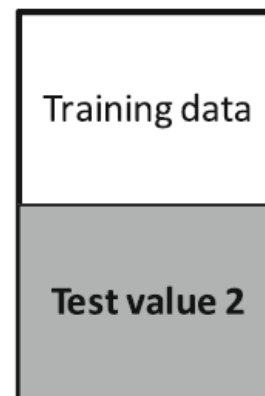
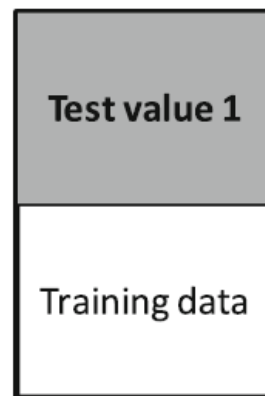
5x2-
partitioning



Iteration 1



Iteration k



Test values
aggregation

`mean(errors)`
`std(errors)`

A more
balanced
partition in
Train/Test

Advantages of 5x2 Cross-Validation

- **Reduced Bias**

Compared to a single train-test split, 5x2 cross-validation reduces the bias in performance estimation as every data point gets to be in the validation set.

- **Robustness**

The two iterations help to assess the stability of the results and reduce the influence of a particular data split.

- **Suitable for Small Datasets**

It is particularly useful when you have limited data, as it maximizes the use of available data for both training and validation



Leave one out! (LOO)

In case you are
underfitting using
k-FCV and 5x2CV

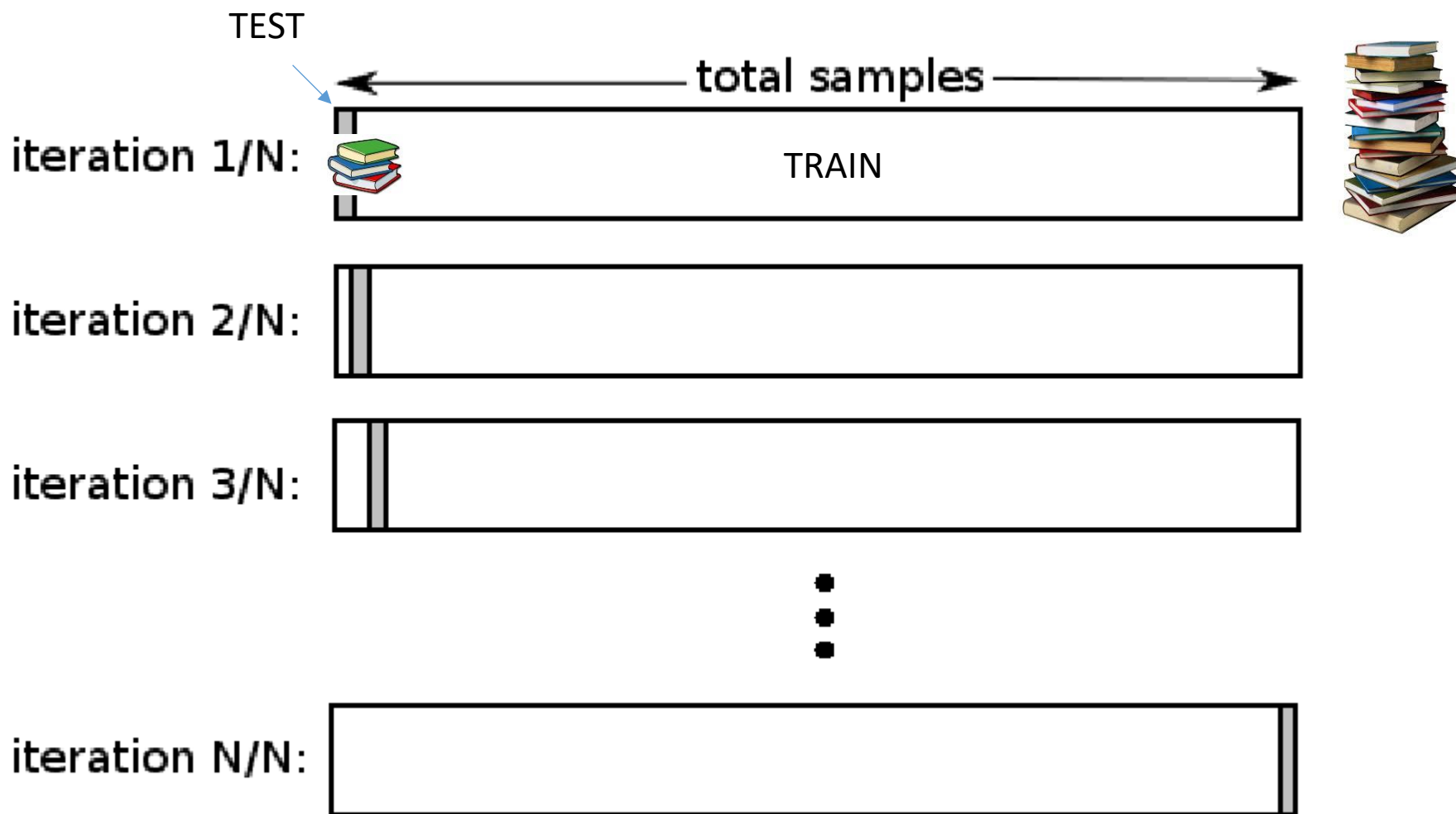


Data Set Partitioning: *Leave one out*

- Is an extreme case of k -FCV $\rightarrow k$ equals the number of examples in the data set
- In each step only one instance is used to test the model whereas the rest of instances are used to learn it.



Data Set Partitioning: *Leave one out*

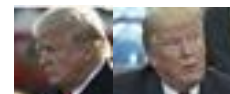




Leave one PERSON out! (LOPO)

In biometrics,
medical screenign, etc.

CRITICAL FOR LARGE/DEEP MODELS

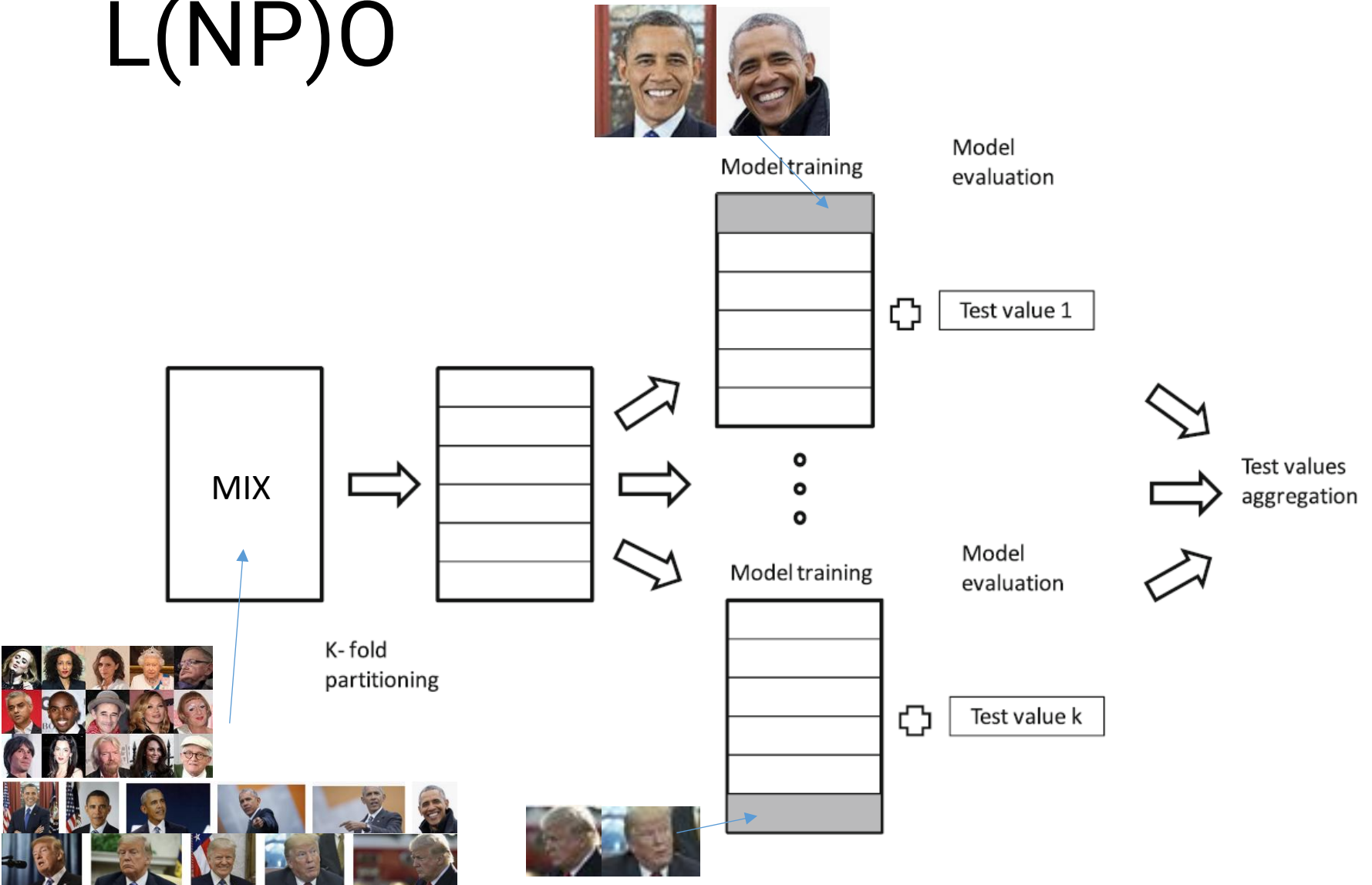


PERSON



LOPO L(NP)O

It's harder now to use
the color of the hair..





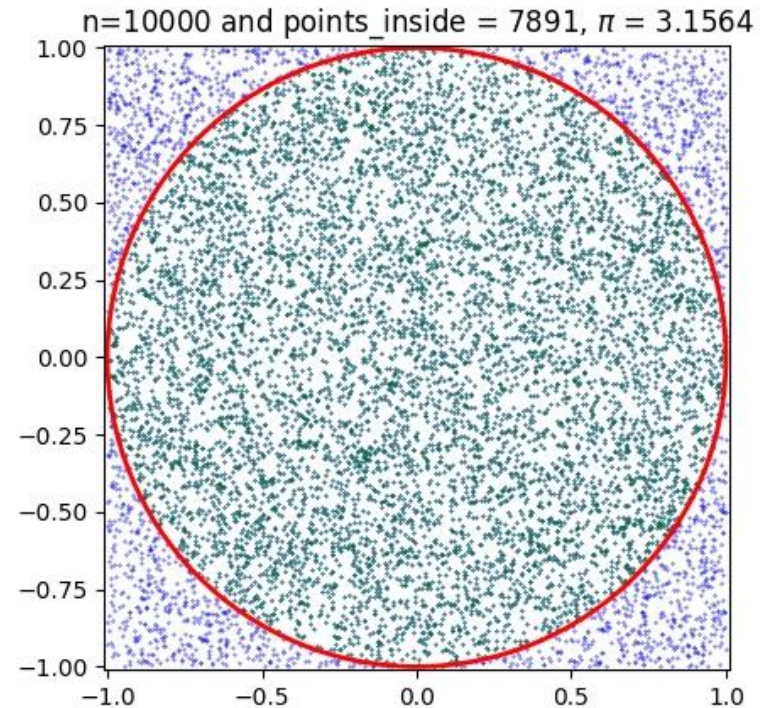
Monte Carlo Validation

(Repeated Random Subsampling Validation)

In finance, medical AI, Signal processing, etc.

The Monte Carlo method

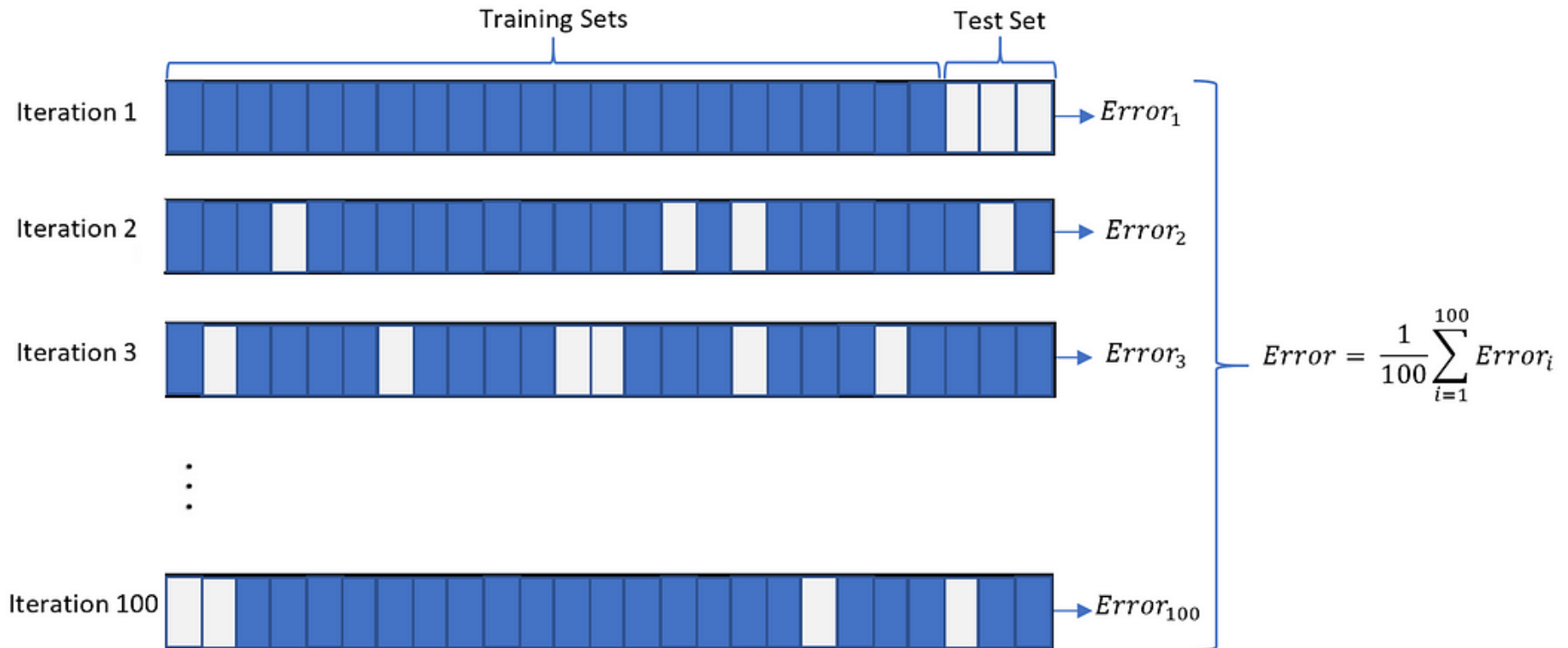
- Monte Carlo methods, or Monte Carlo experiments, are a broad class of computational algorithms that rely on **repeated random sampling** to obtain numerical results.
- The underlying concept is to **use randomness to solve problems that might be deterministic in principle**.



The Monte Carlo method for estimating π by randomly distributing 10,000 points within a square and counting those inside the inscribed circle. With 7,891 points inside, the estimated π value is 3.1564, approximating the true value of 3.1416.

Monte Carlo Validation

(Repeated Random Subsampling Validation)



Monte Carlo Validation (Repeated Random Subsampling Validation)

Monte Carlo Validation, also known as **Repeated Random Subsampling Validation**, is a method for evaluating machine learning models by randomly splitting the dataset **multiple times** into training and test sets.

How it works:

1. The dataset is randomly split into **training and test sets**.
2. The model is trained on the training set and evaluated on the test set.
3. This process is **repeated multiple times**, with a new random split in each iteration.
4. The final model performance is estimated by **averaging the results** across all iterations.

Key points of Monte Carlo Validation



Flexible split ratio:

Unlike k-fold cross-validation, the size of the training and test sets does **not** need to be fixed across iterations.



Repeated randomization:

Each iteration generates a new random partition of the dataset.



Number of repetitions:

The number of iterations is chosen by the user, depending on the desired statistical confidence.

Bias and variance of the errors

In crossvalidation methods (kCV, 5x2CV, LOO, etc.) you test 1 model for every partition. Every model has his error. Hence you get...

Collected errors: $e_1, e_2, e_3, \dots, e_k$

$$\text{Mean CV error} = \frac{1}{k} \sum_{i=1}^k e_i \quad \text{Std CV error} = \sqrt{\frac{1}{k-1} \sum_{i=1}^k (e_i - \bar{e})^2}$$

Imagine testing two classifiers using **5-fold cross-validation**.

Model	Fold errors	Mean error	Variance across folds	Interpretation
Very simple tree	0.31, 0.33, 0.32, 0.34, 0.30	0.320	0.00025	High bias
Very deep tree	0.05, 0.30, 0.08, 0.38, 0.12	0.186	0.02118	High variance

The bias-variance dilemma

- It describes a fundamental tradeoff in machine learning.
- A model must be **complex enough to learn** the real structure of the data, **but not so complex that it learns noise**, accidental patterns, or specific details of the training set.
 - A **too simple model** usually has **high bias, low variance (leading to underfitting)**
 - A **too complex model** usually has **low bias, high variance (leading to overfitting)**
- The bias variance dilemma is about **choosing the model that minimizes the expected error on unseen data while remaining stable across different data splits.**

The bias-variance dilemma

High Bias

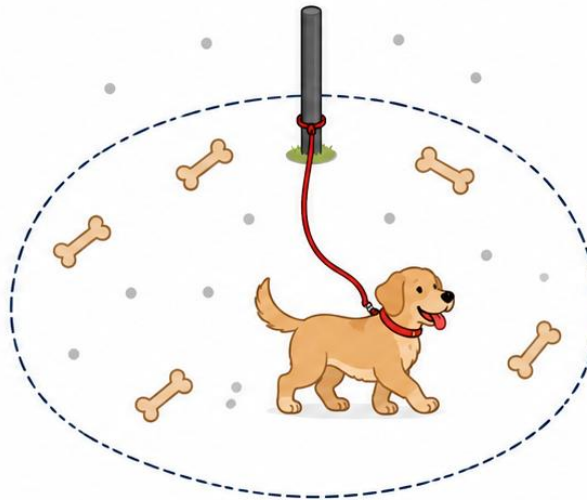
Underfitting



Model too simple.
Misses the true pattern.

Good Tradeoff

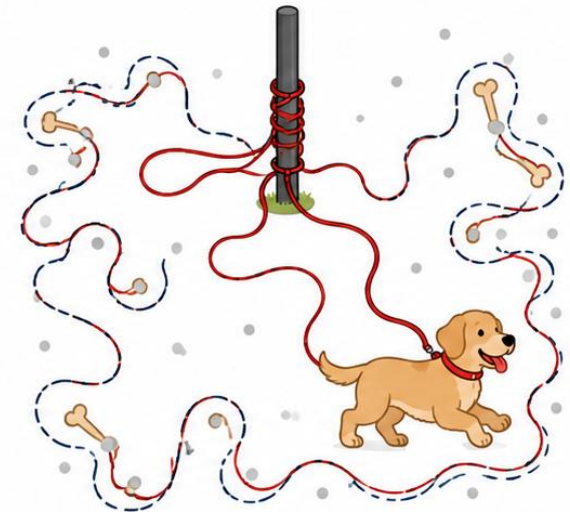
Good Generalization



Balanced complexity.
Captures the overall pattern.

High Variance

Overfitting



Model too complex.
Fits noise and details.



Too simple misses structure. Too complex fits noise. **The goal is the best generalization on unseen data.**

Feature	k-Fold Cross-Validation	5x2 Cross-Validation	Leave-One-Out (LOO)	Leave-One-Person-Out (LOPO)	Monte Carlo Validation
Splitting Strategy	Dataset divided into k equal folds, each used once as test set	Repeated 5 times with 50%-50% train-test split, swapping sets	Each data point used as test set exactly once	Each entire person (subject) left out and used as test set	Random train-test split, repeated multiple times
Train-Test Ratio	1/k of data used for testing in each fold	50%-50% for each iteration	1 sample for testing, all others for training	1 entire subject for testing, all others for training	Flexible
Model Stability	Evaluated across k folds	Repeated 5 times, averaged performance	Unbiased but unstable for small datasets	Robust generalization for subject-based problems	Tested over multiple random splits
Number of Models Trained	k models	10 models (5 splits x 2 swaps)	N models (dataset size)	Number of subjects	Depends on number of repetitions
Computational Cost	Moderate	Moderate	Very high	Very high	Potentially high
Bias-Variance Tradeoff	Balanced tradeoff	Low bias, moderate variance	High variance	Lower variance than LOO	Lower bias, higher variance
Best Use Cases	General-purpose validation	Statistical hypothesis testing	Small datasets	Biometrics, medical, speaker recognition	Large datasets, flexible splits
Limitations	Data must be divisible by k	Fixed split ratio	Does not scale well	Requires subject-based partitions	Some samples never tested, others repeated

Bias measures systematic error due to overly simple models or evaluation strategies.

Variance measures sensitivity to data splits, increasing when models or validation are unstable.

A model with **high bias** typically underfits the data, whereas a model with **high variance** tends to overfit, making the "tradeoff" essential for finding the best generalization.

Key Takeaways

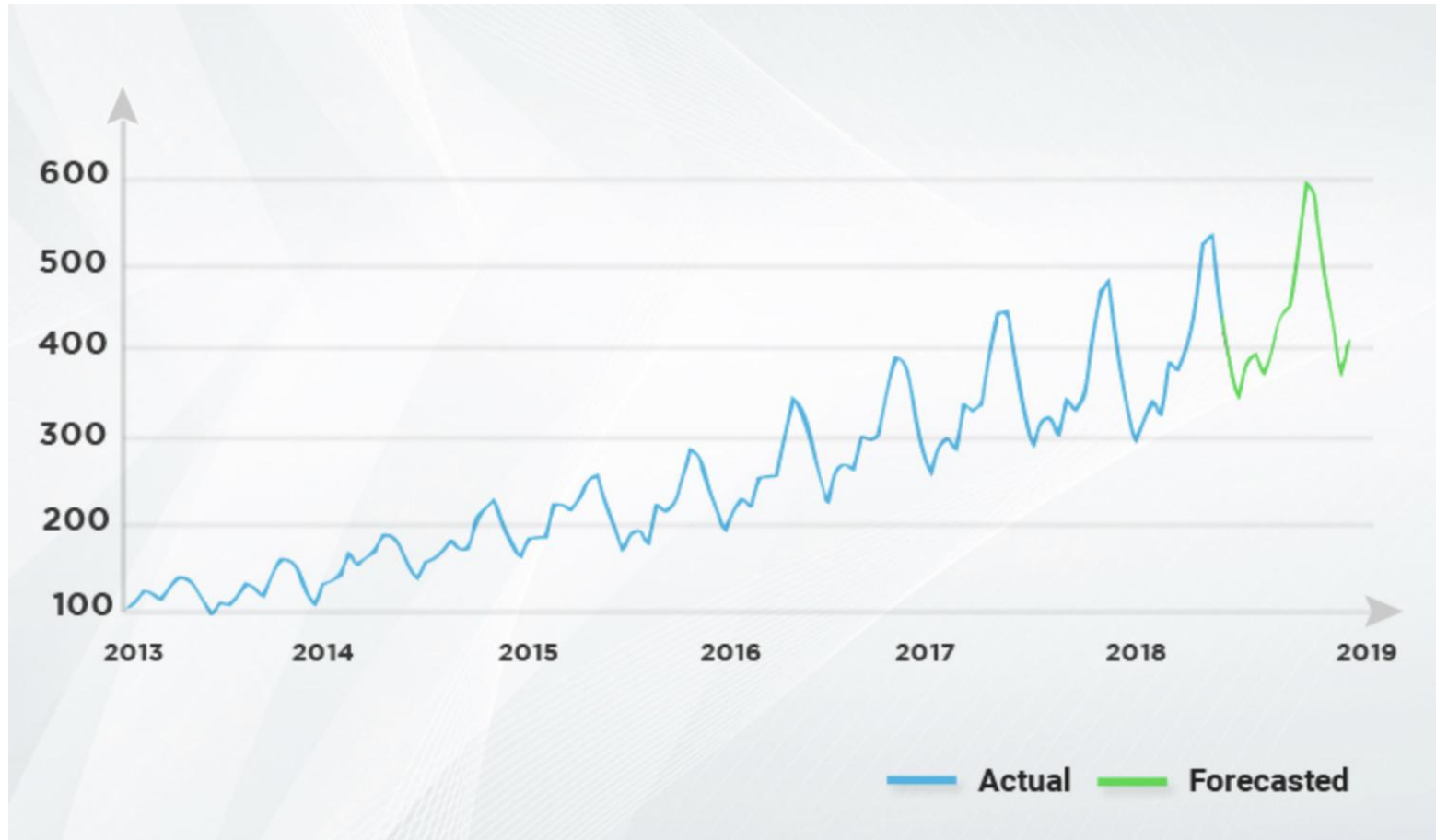
- **Monte Carlo Validation**
offers high flexibility but can be computationally expensive.
- **k-Fold Cross-Validation**
is a well-balanced, widely used method
(consider also Stratified kFCV)
- **5x2 CV**
is useful for comparing models and controlling variance.
- **LOO** (Leave-One-Out)
is suitable for small datasets but can be computationally expensive.
- **LOPO** (Leave-One-Person-Out)
is used for subject-specific generalization problems.



Time series Dataset Validation

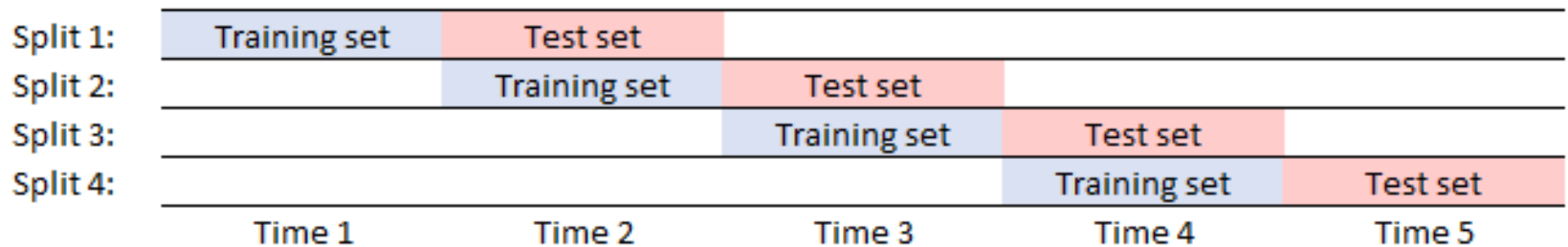
In biometrics, medical screening, forecast, stocks, etc.

The classical problem in time series prediction



A complex task

- Dividing time series dataset into training and test, the dataset can be an error-prone problem
→ Data Leakage can be observed
- One basic system is to simulate the time passing....
- We are wasting a lot of data!

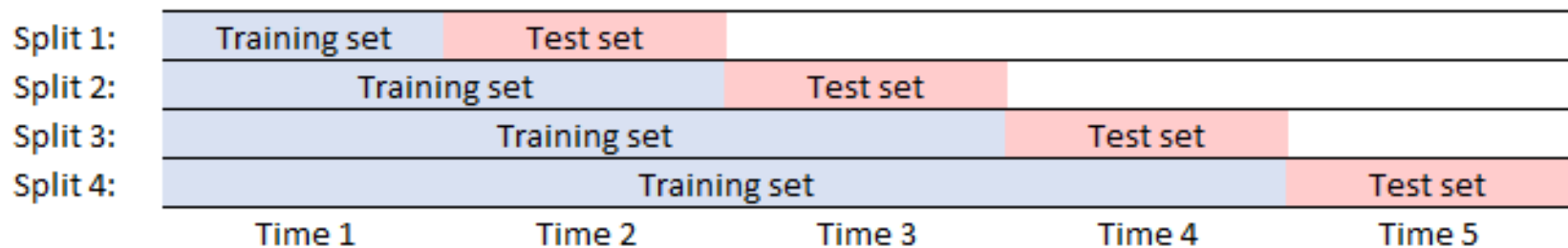


Rolling window validation

A complex task

Expanding window forward

- Dividing time series dataset into training and test, the dataset can be an error-prone problem
→ Data Leakage can be observed
- One basic system is to simulate the time passing....
- Size of the train partitions is different!

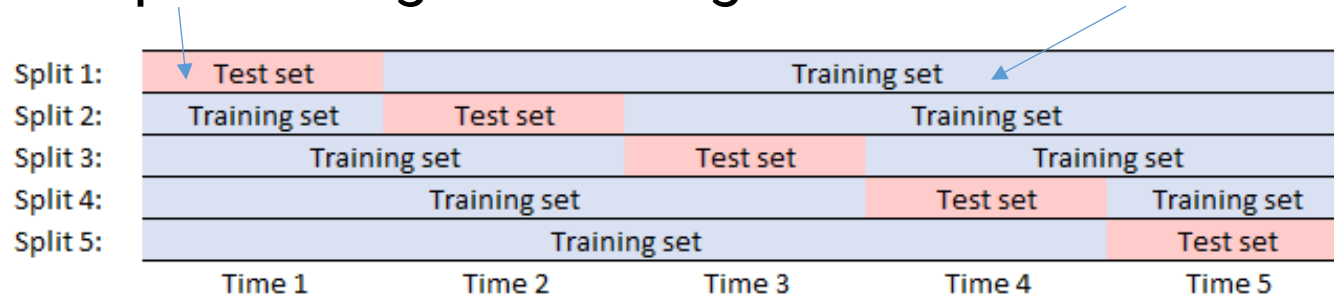


Expanding window forward validation

Dividing time series for cross-validation

On more complex approaches the past-present-future relationship between partitions can be not used (inside the partition the time is normal)

- The model will be able (also) to validate data of the past using as training data in the future



Or also **adding gaps** to make the forecast more challenging
(Gap leave p out)



Main points



- Workflow: optimization
- Dataset Balancing
- Dataset Partitioning
 - Underfitting, Overfitting, Design of the dataset
 - Time series
- Accuracy estimation
 - kFCV, 5x2CV, LOO, LOPO, Monte Carlo
- Two very strong points:
 - DO NOT DESIGN USING THE TEST DATASET
 - 1% ERROR CAN BE GOOD OR BAD